

**A Retrieval-Augmented Generation Framework Using Large Language Models for
Interacting with General Transit Feed Specification (GTFS) Data**

A Thesis Proposal

Presented to the

School of Information Technology

Mapúa University

In Partial Fulfillment

of the Requirements for the Degree

Bachelor of Science in Computer Science

by

John Rey Christian Adao

Vincent Andrei D. Lusung

Janxen Red A. Tan

February 2026

RECOMMENDATION SHEET

This is to certify that I have supervised the preparation of and read the thesis proposal prepared by **John Rey Christian Adao, Vincent Andrei D. Lusung, and Janxen Red A. Tan** entitled **A Retrieval-Augmented Generation Framework Using Large Language Models for Interacting with General Transit Feed Specification (GTFS) Data** and that the said paper has been submitted for final examination by the Oral Examination Committee.

Gloren S. Fuentes
Thesis Adviser

TABLE OF CONTENTS

TABLE OF CONTENTS.....	iii
LIST OF FIGURES	vii
LIST OF TABLES.....	1
Chapter 1	2
1.1. Context of the Study	4
1.2. The Problem, Gap or Opportunity	8
1.3. Research Questions.....	10
1.4. Research Objectives.....	11
1.5. Significance of the Study	11
1.6. Scope and Limitations (or Delimitations).....	13
Chapter 2.....	15
2.1. Large Language Models for Public Transit Data (GTFS) Understanding.....	15
2.2. Program Synthesis for GTFS Query Processing: The TransitGPT Framework	16
2.3. Retrieval-Augmented Generation for Structured Transportation Data.....	19
2.4. Schema-Guided Representation of GTFS Data Using M-Schema	21
2.5 Evaluation Metrics for LLM-Based Data Retrieval and Reasoning Systems.....	23
Chapter 3.....	27
Conceptual Framework.....	27

Data Gathering.....	28
Data Preparation.....	30
Model Setup.....	31
Experimental Setup.....	32
Model Framework.....	33
Model Evaluation.....	34
Chapter 4.....	35
4.1 Benchmark Setup and Evaluation Protocol	35
4.1.1 Benchmark Questions	37
4.2 Results of the TransitGPT Framework	41
4.2.1 Performance Metrics	41
4.2.2 Confusion Matrix Analysis.....	43
4.2.3 Performance by Difficulty Level	44
4.2.4 Performance by Question Category	45
4.2.5 Multi-Table vs. Single-Table Performance.....	46
4.3 Results of the Proposed GTFS-RAG Framework.....	47
4.3.1 Performance Metrics	47
4.3.2 Confusion Matrix Analysis	48
4.3.3 Performance by Difficulty Level	50
4.3.4 Performance by Question Category	51

4.3.5 Sample Evaluation Outputs.....	52
4.3.6 Multi-Table vs. Single-Table Performance.....	54
4.4 Comparative Analysis Between the Two Systems	55
4.5 Retrieval Limitations in the RAG Architecture	58
4.6 Relational Reasoning Challenges in GTFS Data	59
4.7 Advantages of Runtime Code Execution in TransitGPT	60
4.8 Analysis of Architectural Trade-Offs	62
4.9 Implications of the Findings	63
4.10 Connection to Related Literature	64
4.11 Summary of Chapter 4	65
Chapter 5	67
5.1 Summary of the Study	67
5.2 Summary of Findings.....	68
5.3 Answers to the Research Questions.....	69
5.4 Conclusions Based on the Research Objectives	72
5.5 Overall Assessment of the Proposed Framework	73
5.6 Limitations of the Study.....	74
5.7 Importance of the Comparative Findings	75
5.8 Recommendations for Future Researchers	76
5.9 Recommendations for Improving GTFS-RAG Systems	77

5.10 Conclusion	78
REFERENCES	80

LIST OF FIGURES

Figure 1. Conceptual Framework	28
Figure 2. Proposed Conceptual Framework.....	34
Figure 3. TransitGPT Performance Metrics by Difficulty Level.....	45
Figure 4. TransitGPT Per-Question Correctness Grid Across All 75 Benchmark Items.	46
Figure 5. GTFS-RAG System Performance Metrics by Difficulty Level.....	45
Figure 6. GTFS-RAG Per-Question Correctness Grid Across All 75 Benchmark Items.	52
Figure 7. Sample GTFS-RAG Responses — Easy Difficulty Questions.....	53
Figure 8. Sample GTFS-RAG Responses — Medium Difficulty Questions.....	53
Figure 9. Sample GTFS-RAG Responses — Hard Difficulty Questions.....	54
Figure 10. Overall Metric Comparison: TransitGPT vs. GTFS-RAG Across All Four - Evaluation Metrics.....	56

LIST OF TABLES

Table 1. Primary GTFS Files.....	29
Table 2. Complete Benchmark Questions by Difficulty Level (n = 75).....	41
Table 3. Performance Metrics — TransitGPT Framework (claude-sonnet-4-6, n=75)....	41
Table 4. Confusion Matrix — TransitGPT Framework (claude-sonnet-4-6, n=75).....	43
Table 5. TransitGPT Performance by Difficulty Level.....	44
Table 6. Performance Metrics — GTFS-RAG Framework (claude-sonnet-4-6, n=75)....	47
Table 7. Confusion Matrix — GTFS-RAG Framework (claude-sonnet-4-6, n=75).....	48
Table 8. GTFS-RAG Performance by Difficulty Level.....	50
Table 9. Side-by-Side Comparative Metrics: TransitGPT vs. GTFS-RAG (claude-sonnet-4-6, n=75).....	55
Table 10. Difficulty-Level Comparison: Correct Answers per System.....	56
Table 11. Category-Level Performance Comparison: TransitGPT vs. GTFS-RAG.....	57
Table 12. Architectural Trade-Off Summary: TransitGPT vs. GTFS-RAG.....	62

Chapter 1

INTRODUCTION

The General Transit Feed Specification (GTFS) has emerged as a widely adopted standard for representing public transportation schedules and geographic information in a structured, machine-readable format. GTFS datasets consist of multiple interrelated text files that collectively describe routes, stops, trips, schedules, and service calendars. While this standardized format supports consistent representation of transit systems, its relational structure introduces challenges for data interpretation and analysis. Extracting meaningful information from GTFS feeds often requires complex multi-table joins, temporal filtering, and structured querying, which can be difficult to perform without specialized technical knowledge. Recent developments in Large Language Models (LLMs) have demonstrated potential in assisting with complex data reasoning tasks; however, enabling LLMs to effectively operate over structured relational datasets remains a significant research challenge.

Existing approaches for integrating LLMs with structured transportation data typically follow two architectural paradigms. The first approach involves runtime code execution frameworks, where the language model generates executable queries or scripts that interact directly with the dataset. The second approach utilizes Retrieval-Augmented Generation (RAG), where relevant contextual information is retrieved from external data sources and provided to the model during response generation. While both approaches have

shown promise in enabling LLM-based reasoning over structured data, previous studies often evaluate these architectures independently and under different experimental conditions. Consequently, there is limited empirical understanding of their comparative performance when applied to complex relational datasets such as GTFS.

This study proposes a Retrieval-Augmented Generation framework integrated with schema-aware representations derived from the GTFS dataset. The framework first converts raw GTFS data into a relational SQL database and generates M-Schema representations to provide explicit structural descriptions of tables, attributes, and relationships. These representations are embedded and indexed to enable semantic retrieval within a RAG pipeline. When a query is issued, relevant contextual information is retrieved from the indexed schema and dataset fragments and supplied to the language model for response generation. The performance of the proposed framework is evaluated using established machine learning metrics, including accuracy, precision, recall, and F1 score.

The experimental results are compared with an existing runtime code execution framework to assess differences in reasoning performance over structured transit datasets. Through this comparative evaluation, the study aims to examine the strengths and limitations of retrieval-based and execution-based architectures when applied to relational data environments. The findings contribute to the broader understanding of how large language models can be effectively integrated with structured datasets and provide insights into architectural trade-offs for enabling reliable reasoning over complex relational data such as GTFS.

1.1. Context of the Study

Large Language Models (LLMs) have rapidly advanced the field of artificial intelligence, demonstrating strong capabilities in natural language understanding, reasoning, and knowledge generation. These models have been successfully applied to a wide range of tasks including question answering, information retrieval, and conversational systems. However, a significant challenge remains in LLMs' ability to effectively interact with structured data environments. In most cases, structured data sources are not contained in a single unified environment but are fragmented across multiple files that describe various facets of a system. As a result, understanding the context requires interpreting inter-file relationships and combining pertinent information to understand a given data environment. LLMs are most likely to struggle with interpreting fragmented data sources, especially when it is not possible to explicitly describe inter-file relationships in natural language. In addition, large datasets cannot be injected into a model's context due to input length restrictions and the risk of overwhelming the model with excessive information. As a result, there is a growing need to develop methodologies that enable LLMs to effectively retrieve and interpret pertinent information while preserving inter-file relationships. Such problems are especially common in data-intensive environments that are naturally fragmented, such as public transportation systems that rely on the General Transit Feed Specification (GTFS).

The General Transit Feed Specification (GTFS) is an open and standardized data format designed to represent static public transportation information in a structured and machine-readable form. It consists of a collection of comma-separated text files, each describing a specific operational component of a transit system, including routes, stops,

trips, stop times, service calendars, and fare attributes. These files are organized under a relational schema, where entities are interconnected through primary and foreign key relationships. Collectively, the dataset models the logical structure of public transportation operations, capturing both spatial and temporal aspects of transit services. GTFS plays a critical role in standardizing how transportation data is published, exchanged, and analyzed. Defining a uniform schema enables consistent representation of transit networks across different geographic regions and agencies.

However, one major challenge in working with GTFS data is its inherent structural complexity. A single feed can consist of dozens of text files, encompassing hundreds of fields that are interconnected through primary and foreign keys and governed by temporal, operational, and logical constraints. Fields vary in type and requirement—some are mandatory, others optional, and some conditionally present depending on the values of other fields—while the data itself spans times, stop coordinates, route shapes, identifiers, counts, fares, and color codes. Understanding the relationships among these diverse elements is essential for meaningful analysis. Devunuri et al. [1] showed that popular LLMs, such as GPT-3.5-Turbo and GPT-4, can partially address this challenge by answering questions about the GTFS specification with reasonable accuracy. A second challenge arises when an analyst seeks specific information about a transit agency or service. Even with a solid understanding of GTFS, extracting answers typically requires coding or manipulating data with spreadsheets to join tables, apply temporal constraints, or filter relevant records. Lewis et al. [2] introduced the Retrieval-Augmented Generation (RAG) framework, in which a retriever first searches for relevant external data based on a

user query, and the retrieved information is explicitly included in the prompt given to the language model.

It is important to note that Retrieval-Augmented Generation (RAG) can, in principle, pass relevant portions of a GTFS feed to an LLM in order to answer specific queries. While RAG can retrieve the feed in chunks [2][3], a key limitation arises from the interrelated nature of GTFS files—no single file contains all the information about a given route, stop, or trip, making comprehensive retrieval and reasoning over multiple files challenging. To address this gap, the *M-Schema framework* is introduced, which constructs a standardized, semi-structured representation of a database schema optimized for LLM consumption by extracting tables, columns, relationships, and constraints into a consistent format [4]; this enables more accurate reasoning and query generation over relational data such as GTFS feeds.

While RAG and M-Schema provide promising mechanisms to improve LLM reasoning over GTFS data, several limitations remain that motivate further innovation. First, the interconnected nature of GTFS files presents a structural challenge stops, trips, and routes are spread across multiple files, and temporal constraints often span several tables. This means that even when relevant subsets are retrieved using RAG, the model must still correctly align and join these pieces, a task that can easily lead to inconsistencies or incomplete responses if not carefully managed. Second, many transit-related queries require numerical computation or conditional logic, such as calculating the shortest travel time, identifying overlapping trips, or summarizing weekend service frequencies. LLMs, while proficient at generating natural language and interpreting instructions, are prone to errors in precise arithmetic or multi-step data manipulation [5]. Without a programmatic

mechanism to enforce correctness, these errors can propagate and reduce the reliability of the system.

Furthermore, context length limitations remain a practical obstacle. Public-facing LLMs cannot ingest entire GTFS feeds due to their size, making selective retrieval essential. Even with chunked retrieval via RAG, some queries may require information from multiple interdependent chunks, highlighting the need for schema-aware reasoning to reconstruct relationships accurately. Here, M-Schema proves valuable by providing the LLM with an explicit representation of the database structure, including relationships between tables and constraints, enabling more precise query synthesis. By combining retrieval with schema awareness, the model is better equipped to perform reasoning over structured transit data than relying solely on its pretrained knowledge.

Nevertheless, there is a remaining gap: existing solutions focus primarily on retrieving relevant data or encoding schema structure, but they do not fully integrate reasoning, retrieval, and programmatic execution in a unified pipeline. This limitation motivates approaches like TransitGPT, which couples LLMs with program synthesis. In such frameworks, the model not only interprets user queries in natural language but also generates executable code that interacts directly with GTFS tables, performs calculations, and respects relational and temporal constraints. By integrating schema guidance, selective retrieval, and automated code generation, this approach aims to provide highly accurate, context-aware, and operationally correct responses to complex transit questions, overcoming the primary weaknesses of standalone LLM or RAG-based methods.

Finally, implementing such a system in practice raises additional considerations, such as latency, error handling, and verification of generated code. Each retrieval and execution step introduces potential delays and failure points, particularly when multiple tables must be joined or constraints applied. Therefore, part of the proposed framework also emphasizes robust backend integration, where retrieved data is validated against schema rules and computed outputs are verified before being presented to the user. This ensures that responses are not only syntactically correct but also semantically faithful to the underlying GTFS dataset, providing transit analysts and end users with reliable, actionable information.

1.2. The Problem, Gap or Opportunity

Despite the widespread availability and standardized structure of the General Transit Feed Specification (GTFS), effectively leveraging these datasets for meaningful analysis remains a persistent challenge. GTFS feeds consist of dozens of interrelated text files representing stops, routes, trips, stop times, and other operational details, requiring extensive data manipulation and multi-table joins to answer even moderately complex queries. Traditional methods, such as manual processing or spreadsheet-based analysis, are labor-intensive, error-prone, and infeasible for non-technical users. Although GTFS is widely used for planning, scheduling, and performance evaluation, the complexity of its relational schema continues to hinder automated or scalable query processing [6][7]. As previous studies have found, even experienced analysts often struggle to retrieve, align, and interpret data across interdependent files without specialized software tools or coding expertise. [8]

Recent research has investigated the use of Large Language Models (LLMs) to address aspects of this challenge. For example, Devunuri et al. [9] introduced benchmarks such as “GTFS Semantics” and “GTFS Retrieval” to evaluate how well LLMs can understand and extract information from GTFS feeds, finding that LLMs like GPT-4 achieve high performance on simple queries but degrade on more complex retrieval tasks. Similarly, TransitGPT demonstrates that LLM-generated executable code can be used to interrogate GTFS data on a server, enabling users to obtain structured results without deep technical knowledge. These approaches highlight the potential of combining natural language interfaces with AI to democratize access to transit datasets.

A clear gap remains in the lack of a unified comparative framework that systematically evaluates Retrieval-Augmented Generation (RAG) architectures and runtime code execution architectures under the same dataset, task conditions, and evaluation of metrics. Existing studies typically focus on improving a single component—retrieval of accuracy, schema representation, or program synthesis—without conducting controlled backend-level comparison across architectural paradigms [10].

Furthermore, key evaluation dimensions such as task accuracy, faithfulness to structured data, retrieval effectiveness, latency, and failure rate are not consistently analyzed together [11]. As a result, there is limited empirical evidence clarifying the architectural trade-offs between retrieval-grounded generation and executable query synthesis when applied to relational transportation datasets. This gap presents an opportunity to conduct a structured empirical comparison of these competing architectures within a controlled GTFS environment. By evaluating both frameworks under standardized

metrics, this study aims to determine their relative strengths, limitations, and performance characteristics in structured transit data reasoning.

1.3. Research Questions

The research questions were formulated to guide the study toward achieving the stated objectives and to ensure that the study addresses both the technical and practical aspects of interacting with GTFS data using AI-driven frameworks.

Specifically, the study seeks to answer the following research questions:

- (1) How can raw GTFS data be effectively transformed into a relational SQL database while generating M-Schema representations for each table to facilitate LLM-based reasoning?
- (2) How can a Retrieval-Augmented Generation (RAG) pipeline integrated with a Large Language Model be developed to interact with GTFS data and respond to context-specific transit queries?
- (3) How effective is the proposed RAG + LLM framework in terms of context recall, accuracy, precision, and F1 score when answering queries over GTFS datasets?
- (4) How does the proposed framework compare with an existing framework in terms of context recall, accuracy, precision, and F1 score, and what improvements or limitations are observed?

1.4. Research Objectives

The primary goal of this study is to develop an AI-driven framework capable of efficiently processing and reasoning over complex transit datasets, specifically GTFS, to support accurate, context-aware query generation.

The objectives were formulated to address the following research questions:

- (1) Design a preprocessing pipeline to convert raw GTFS data into a relational SQL database and generate M-Schema representations for each table.
- (2) Develop a RAG pipeline integrated with an LLM framework to enable interaction with GTFS data.
- (3) Evaluate the proposed framework using metrics such as context recall, accuracy, precision, and F1 score.
- (4) Compare the proposed framework with an existing system based on context recall, accuracy, and F1 score.

1.5. Significance of the Study

This study is significant to various stakeholders in the fields of *Computer Science, public transportation, and urban development, particularly within the Philippine context.*

From a **computer science and artificial intelligence perspective**, the study contributes to advancing the capabilities of Large Language Models in reasoning over complex, structured datasets. By integrating Retrieval-Augmented Generation (RAG), schema-aware frameworks such as M-Schema, and program synthesis approaches exemplified by TransitGPT, the research addresses challenges related to multi-table relational reasoning, temporal and operational constraints, and query-specific data

manipulation. These contributions expand the frontier of AI research by demonstrating methods for combining retrieval, schema representation, and programmatic execution to solve real-world, structured-data problems, which can also be applied to other domains like finance, healthcare, and logistics that rely on relational datasets.

From a **public transportation standpoint**, this study provides practical tools for improving the accessibility, efficiency, and reliability of transit data analysis. Transit agencies, planners, and commuters can benefit from automated systems capable of responding to complex queries, such as identifying optimal routes between locations at specific times, estimating delays based on service schedules, or analyzing the distribution of trips and stops across days of the week. These capabilities reduce the dependency on manual data processing, minimize human error, and accelerate decision-making processes, enabling agencies to respond more quickly to service disruptions or emerging transit demands. Additionally, commuters can access accurate, context-aware information to make better travel decisions, which can improve satisfaction, reduce waiting times, and promote the use of public transit.

In the context of **urban development and policy**, the study supports evidence-based planning and operational improvements in transportation networks. By enabling efficient and accurate reasoning over GTFS data, the proposed framework provides urban planners and policymakers with actionable insights into network connectivity, service gaps, and infrastructure utilization. This is particularly relevant in rapidly growing metropolitan areas in the Philippines, such as Metro Manila, where traffic congestion, uneven transit coverage, and commuter demand fluctuations create complex mobility challenges. Insights generated from structured GTFS analysis can inform decisions on

route optimization, allocation of resources, service expansion, and accessibility improvements for underserved areas, thereby supporting inclusive urban development and sustainable mobility initiatives.

Furthermore, the study has **broader societal implications**. Improved access to reliable and structured transit data fosters transparency, encourages civic engagement, and empowers researchers and advocacy groups to analyze and propose improvements to public transportation systems. It also contributes to economic efficiency by reducing operational inefficiencies and enabling data-driven investment in transit infrastructure. By demonstrating how AI-driven systems can bridge the gap between complex data and actionable insights, the research highlights the potential of emerging technologies to create smarter, more resilient urban environments that better serve the needs of citizens.

Overall, this study bridges **technical innovation with practical application**, demonstrating how advances in AI can directly support public transportation planning, urban mobility, and policy formulation. It underscores the importance of combining retrieval, schema representation, and automated reasoning to transform large, complex datasets into actionable knowledge, providing benefits not only to researchers and agencies but also to the commuting public and the broader society.

1.6. Scope and Limitations (or Delimitations)

To ensure feasibility within time, resource, and technical constraints, the following limitations are acknowledged:

- i. The study is restricted to static GTFS data and does not include real-time updates such as live train positions, delays, or service interruptions.

- ii. The geographic coverage excludes informal and semi-formal transport modes (e.g., jeepneys, tricycles, and other non-rail public transport) due to the lack of standardized and reliable digital transit data.
- iii. The system does not account for unannounced service disruptions, emergency schedule changes, or sudden fare adjustments that occur after the most recent data update.
- iv. The proposed AI framework focuses exclusively on GTFS data and does not integrate other transportation-related datasets, such as traffic sensor data, weather information, or passenger flow statistics, which could affect route optimization and service analysis.
- v. Evaluation emphasizes metrics such as context recall, precision, F1 score, and accuracy. User experience, latency, or system scalability under heavy query loads is not formally assessed.
- vi. The study focuses specifically on the Manila GTFS dataset (manila.zip), which serves as the primary source for building and evaluating the proposed framework.

Chapter 2

REVIEW OF RELATED LITERATURE & STUDIES

The study reviews existing literature on the utilization of large language models within structured data environments, with a focus on transportation systems that utilize the General Transit Feed Specification (GTFS). The existing studies suggest that large language models can process natural language queries and interact with complex datasets, thus providing new avenues for data retrieval and reasoning. However, the fragmented structure of GTFS data introduces challenges for models that must reason across multiple interconnected files. The study therefore examines literature on program synthesis approaches, retrieval-augmented generation architectures, and schema-guided data representations in order to establish the theoretical foundation for integrating large language models with retrieval mechanisms and M-schema-based representations of GTFS data.

2.1. Large Language Models for Public Transit Data (GTFS) Understanding

Public transportation systems have been explored as potential applications of Large Language Models (LLMs), considering their ability to process complex data sets as well as queries based on natural language. In the context of transportation management, LLMs have been observed to be capable of interacting with structured transportation data sets like the General Transit Feed Specification (GTFS), which includes transportation routes, stops, schedules, as well as service operations. Structured public transit datasets such as

GTFS are widely used in transportation planning but remain challenging to interpret and process due to their fragmented structure and complex relationships between files [12].

Within a research study conducted by Jonnala et al. [13], the applicability of LLMs in the context of the public transportation system in San Antonio was assessed. According to the results obtained in the study, it can be stated that LLMs have the ability to comprehend transportation schedules and associated services. However, there are certain challenges that need to be considered in the context of the application of LLMs in structured transportation systems. For instance, there is a considerable chance of inaccuracies to occur in the results generated by the model. In addition, there is complexity associated with the interpretation of relational transportation data. According to the results obtained in the research study conducted in the context of the San Antonio transportation system, it can be stated that GPT-4o models have the ability to attain high accuracy in attribute mappings and information retrieval from the GTFS feed. However, the accuracy may reduce with regard to handling categorical mappings or complex queries that may require the handling of multiple files. Thus, it is apparent that there is a need to enhance the understanding of public transportation data sets.

2.2. Program Synthesis for GTFS Query Processing: The TransitGPT Framework

Recent research has focused on the use of large language models in the interaction with structured data sets in the field of transportation, specifically with the General Transit Feed Specification (GTFS). A major methodology in this area is the TransitGPT framework, as proposed by Devenuri and Lehe [1]. The framework uses program synthesis to process natural language queries about a particular public transportation system. Instead of directly producing text as a final response to the input query, this framework uses large

language models to generate Python code that can interact with GTFS data sets. The overall process of the proposed framework is based on a structured pipeline that includes moderation, code generation, execution, and summarization. In this pipeline, the input query to the model is first validated before being sent to a primary model that generates Python code that can interact with GTFS data structures. Once the code is generated, it is executed in a controlled environment on a server, and the final outputs are converted to natural language to be explained to the end user. In order to effectively process the data in the GTFS data set, this model transforms the text files in the GTFS data set to a structured pandas data frame using preprocessed feed objects.

Previous research conducted by Devunuri et al. [9] focused on exploring the basic proficiency of large language models in interacting with General Transit Feed Specification (GTFS) through developing two benchmarks, namely "GTFS Semantics" and "GTFS Retrieval." Through the research, it was evident that large language models, such as GPT 3.5 and GPT 4, had basic proficiency in understanding conventions related to transit data, achieving accuracy rates of 59.7%, and 73.3%, respectively. However, it was evident from the study that, despite large language models having basic proficiency in interacting with GTFS, there was a tendency for hallucinations and logical fallacies. Notably, there was a marked improvement in information extraction when program synthesis was utilized. The research paper highlighted the potential and limitations of large language models in interacting with GTFS and led to the creation of TransitGPT, a program synthesis-based model for addressing intricate reasoning across multiple GTFS feeds.

One of the major features of the TransitGPT model is the use of program synthesis to handle the complexity associated with GTFS datasets. GTFS datasets include a set of

interconnected files that store data regarding routes, stops, trips, and schedules. The complexity associated with the data requires multi-step reasoning and data manipulation to answer user queries. The authors developed a detailed prompt to guide the language model in producing the correct code. The prompt describes the model's purpose, task requirements, and the type of data used within the GTFS schema. The model also uses dynamic few-shot prompting, where the most relevant user queries are selected based on TF-IDF similarity for the provision of contextual information to the model. Additionally, the model uses custom Python functions that allow the model to perform certain operations, such as fuzzy matching of stop names and route names. The model uses these tools to handle user queries with various variations, including spelling mistakes and queries based on nearby geographical features instead of the GTFS stop names.

Despite these capabilities, the authors outline various limitations of the proposed framework in their study. Firstly, the study does not use Retrieval-Augmented Generation because the GTFS data is spread across different files that are connected with each other, and hence document retrieval is not sufficient to handle most of the queries related to transit information. Therefore, the authors use a program synthesis approach that allows the model to carry out different operations that are beyond the scope of direct reasoning the model. These operations include the combination of different GTFS tables, filtering schedules, and carrying out different computations that are necessary to process complex queries. Moreover, the proposed framework includes an error-handling mechanism that makes the process more robust and ensures that execution errors are returned to the model to refine the generated code until a correct result is obtained. Although the proposed framework performs satisfactorily in handling complex GTFS-based queries, it can currently handle

static GTFS feeds and does not support dynamic or real-time information. However, the proposed framework provides a promising basis on which different researchers can work to improve the interaction between large language models and the GTFS dataset.

2.3. Retrieval-Augmented Generation for Structured Transportation Data

Retrieval-Augmented Generation (RAG) represents an important development in natural language processing by integrating the parametric knowledge stored within large language model weights with non-parametric external knowledge sources. The concept was introduced by Lewis et al. [2], who proposed a hybrid architecture that combines a retrieval mechanism with a text generation model. In this framework, a retriever performs a query over a document index or vector database, after which the generator conditions its output on the retrieved information. By enabling models to access explicit external memory during the inference process, RAG reduces the limitations associated with relying solely on internal model parameters. Empirical findings from this work suggest that RAG is particularly effective for knowledge-intensive tasks such as open-domain question answering and reasoning, where models must retrieve accurate information from large external knowledge bases.

The traditional Retrieval-Augmented Generation (RAG) architecture is comprised of three primary components: the indexing module, the retrieval module, and the generation module, as discussed by Gao et al. [3]. In the indexing module of the RAG model architecture, external information is converted to vector-based representations that can be archived in high-dimensional databases. Upon receiving a query, the retrieval module compares its semantic similarity with the stored information and generates a score on how similar it is to the query. The information is then fed into the generation module,

enabling the language model to generate responses based on external information. This enables RAG-based systems to dynamically feed domain-specific information into language models rather than relying on pre-trained information during inference. This provides a feasible approach for integrating dynamic and domain-specific information into language models.

Recent studies have been able to extend RAG models to accommodate scenarios where there is fragmented or structured knowledge, where the required information is spread out over several data sources or document segments. Li et al. [14], for example, proposed the StructRAG model, which improves the performance of the reasoning task through the use of structured representations of the scattered textual data, such as tables and graphs, which can represent relationships between disparate data pieces. This can eliminate the noise often present in chunk-based systems. From the studies conducted in this research, it has been observed that structured-based retrieval can lead to better performance in tasks that require global reasoning across scattered data elements. Such performance improvement can be observed as the complexity of the IR task increases.

In the context of domain-specific applications, RAG has been extended to incorporate ontology-grounded retrieval mechanisms that provide a foundation for knowledge based on pre-defined conceptual frameworks. Sharma et al. [15] proposed an ontology-grounded RAG methodology that incorporates knowledge representations in the form of structured representations like hypergraphs, in which the structure of the hyperedges identifies clusters of associated factual knowledge that provide a foundation for the preservation of inter-entity relationships that may not be maintained in the context of conventional forms of retrieval. Such an architectural design provides a foundation for

reasoning in accordance with domain-specific structural constraints that require the maintenance of factual consistencies. Such methodologies have particular significance in the context of structured data systems like transportation systems and the General Transit Feed Specification (GTFS), in which relevant knowledge is distributed across multiple interconnected files and attributes.

2.4. Schema-Guided Representation of GTFS Data Using M-Schema

The Text-to-SQL paradigm has been designed to transform natural language-based queries into Structured Query Language (SQL), thus minimizing the hurdles for non-technical users to access complex relational databases [16]. As part of this process, the process of schema linking is recognized as the core component in which the model is able to recognize and map the natural language-based reference to any entity in the question with the corresponding database schema [17]. This process is significant in order to enable the Text-to-SQL model to process queries from different domains, especially in cases where the natural language-based representation in the question does not directly correlate with the technical terms used in the database schema.

The ever-increasing level of complexity in current databases creates reasoning challenges that require models to process intricate relational information to effectively generate queries. Recent research has shown that failures in linking schemas are among the largest error types in current Text-to-SQL models, particularly in cases where the model is unable to make necessary discriminations in ambiguous column names or cannot determine the relevant entities to effectively process a query [17]. This is particularly compounded by the increased number of tables and relationships in the database, forcing the model to reason in an intricate relational search space and increasing the chances of logical error in

the process [18]. These challenges clearly point to the limitations of using implicit reasoning in large language models to process structured and relational information in databases.

To mitigate these issues, various schema-based representations have been proposed that distinguish the process of schema linking from logical parsing. For example, various approaches like RESDSQL have been proposed that incorporate mechanisms to rank and filter relevant schema items before the actual process of query generation [18]. This allows the model to focus on a limited set of relevant database items. For example, in the context of text-to-SQL generation using large language model (LLM)-based representations, recent studies have shown that the overall accuracy of generated SQL queries can be significantly enhanced by incorporating the relationships between entities in the database [19]. Such a principle can be applied in the context of transportation-based datasets like the General Transit Feed Specification (GTFS), in which the overall information is distributed across multiple relational files.

However, previous research on the application of LLM-based transit query systems has found fragmentation to be a major limitation in using retrieval-based models for GTFS data. In the TransitGPT model, the authors recognize this limitation in the application of Retrieval-Augmented Generation (RAG), as this model was not used in their framework due to the fact that the data in the GTFS is not contained in a single consolidated document, but is instead fragmented across multiple, albeit interconnected, files. This, in turn, limits the application of the simple retrieval-based model, as the data in the GTFS is not conducive to answering multiple queries, which involve joining, filtering, and computation across multiple tables [1]. As such, in this study, a schema-guided representation is used,

whereby the GTFS data is represented in the form of an M-schema JSON data structure. This representation minimizes fragmentation in the representation of the relationship between GTFS data and, in turn, improves the relational roadmap presented to the language model.

2.5 Evaluation Metrics for LLM-Based Data Retrieval and Reasoning Systems

Evaluation is a critical process in the identification of system limitations as well as in the development of systems that can efficiently process scenarios in the real world, as benchmarking tests often do not fully account for the critical reasoning and emergent properties that can be observed in large language models [20]. In particular, this is critical in systems that require reasoning as well as data retrieval, as the performance of the system not only depends on its ability to output correct results but also to interpret the correct contextual information. Such consideration is particularly critical in systems that need to be deployed in the real world, as errors in reasoning can have critical consequences in critical domains like healthcare, finance, and transportation.

Currently, evaluation of Retrieval-Augmented Generation (RAG) systems is based on grounding and contextual coherence. For instance, faithfulness is used for evaluating whether the response is grounded based on the evidence provided by the system and not based on its internal knowledge [21]. The complexity of evaluating Retrieval-Augmented Generation (RAG) systems arises in a noisy environment in which the system retrieves contexts with contradictory information. In this situation, the system has to show its ability to be faithful to the evidence provided rather than reaching conclusions without evidence [22]. By analyzing the system's precision in the retrieval process, its recall in the retrieval process, and its faithfulness to the evidence provided, it is possible to determine whether

the generated response comes from the evidence provided or the system's hallucinated reasoning.

Generally, the comparative evaluation of the frameworks of retrieval and reasoning, based on the performance of the machine learning models, is based on classical machine learning performance metrics derived from the confusion matrix, which represents the prediction results in terms of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN) [23]. These values are used to represent the performance of the models using the following mathematical equations:

$$\mathbf{Accuracy} = \frac{(TP+TN)}{(TP+TN+FP+FN)}$$

$$\mathbf{Precision} = \frac{TP}{(TP+FP)}$$

$$\mathbf{Recall} = \frac{TP}{(TP+FN)}$$

$$\mathbf{F1-Score} = \frac{2 \cdot (Precision \cdot Recall)}{(Precision + Recall)}$$

In this context, accuracy refers to the overall proportion of correct predictions made in all cases, although it should be noted that this figure can be quite misleading, especially when dealing with imbalanced datasets. Precision, on the other hand, evaluates the trustworthiness of positive class predictions, where high precision means that the system does not make false positives. Recall, on the other hand, evaluates the ability of the system to capture all cases, where high recall means that the system has captured the class comprehensively. The F1 score combines precision and recall, where the harmonic mean is used, thus ensuring that the performance of the system is not artificially inflated.

To achieve proper evaluation, it is important to incorporate these metrics in overall evaluation models that can accommodate the unique characteristics of large language model (LLM) architectures. Recent studies emphasize the importance of carefully curated datasets, evaluation models, and module-level evaluation as a part of the overall evaluation process of LLM systems [23]. These models can help in evaluating different modules of the model, including the data retrieval and reasoning modules, while at the same time addressing the concerns of prompt sensitivity and non-determinism. By using these metrics, it is possible to develop performance benchmarks and conduct comparative analyses of different data retrieval and reasoning models based on LLM systems.

In comparative model evaluation, higher values of these metrics indicate better performance of the model; however, it is dependent upon the particular task and dataset used. As emphasized in the paper, it is not recommended to interpret evaluation metrics in isolation; instead, it is dependent upon the operational context of the system. In this paper, the same dataset is used as in the basic framework for evaluation; hence, it is easier to compare the TransitGPT framework and the proposed framework with RAG and M-schema representation. By using the same dataset, it is ensured that differences in evaluation metrics between the two frameworks can be attributed only to differences in architecture and not to differences in data set distribution. Hence, it is more reliable for comparing enhancements in data retrieval and reasoning capabilities of the proposed framework while interacting with GTFS data sets.

The reviewed literature highlights the potential for interacting with structured transportation data using large language models, while also pointing to some of the limitations in terms of managing the fragmented data structure of GTFS data. The

methodologies, such as program synthesis and retrieval-based generation, provide scope for reasoning and knowledge retrieval, but may also face some limitations in terms of managing the relational data distributed across multiple files. The limitation is addressed in this study by using the combination of retrieval-based generation and M-schemas in the context of GTFS data, thereby providing more scope for the interaction of language models with structured transportation data.

Chapter 3

METHODOLOGY

This chapter presents the research design and methodological approach employed to develop and evaluate an AI-driven framework for interacting with GTFS data. It outlines the processes involved in collecting, preparing, and structuring the dataset, as well as the construction of the retrieval-augmented generation (RAG) pipeline integrated with large language models (LLMs). Furthermore, the chapter details the model architecture, data preprocessing steps, retrieval and generation modules, and the performance evaluation metrics used to assess the framework.

Conceptual Framework

Currently, transportation applications often lack effective methods to interact with GTFS data. Generative artificial intelligence offers a solution to address these limitations. Devunuri and Lehe [1] proposed a framework called TransitGPT, which directly interacts with GTFS data to process and interpret transit information. GTFS is a standardized format that provides structured data on transit systems, including routes, schedules, stops, and trip details, stored on a server as the authoritative source of transit information.

The process begins with the user submitting a natural language query related to transit schedules, routes, or stops. The query is first validated to ensure it falls within the domain scope. Once validated, it is combined with a system prompt containing schema-aware descriptions of GTFS tables and relationships, as well as a few-shot reasoning guide.

This prompt-engineering step allows the framework to interpret the semantic structure of the transit data without direct retrieval mechanisms.

The model component of the framework uses advanced LLMs such as GPT-4o and Claude-3.5-Sonnet, evaluated using a benchmark dataset of 100 transit-related tasks. The results show that TransitGPT enhances accessibility and usability of transit data by enabling users to interact with complex datasets conversationally, without requiring technical knowledge of GTFS or programming. Figure 1 illustrates the system flow of the conceptual framework.

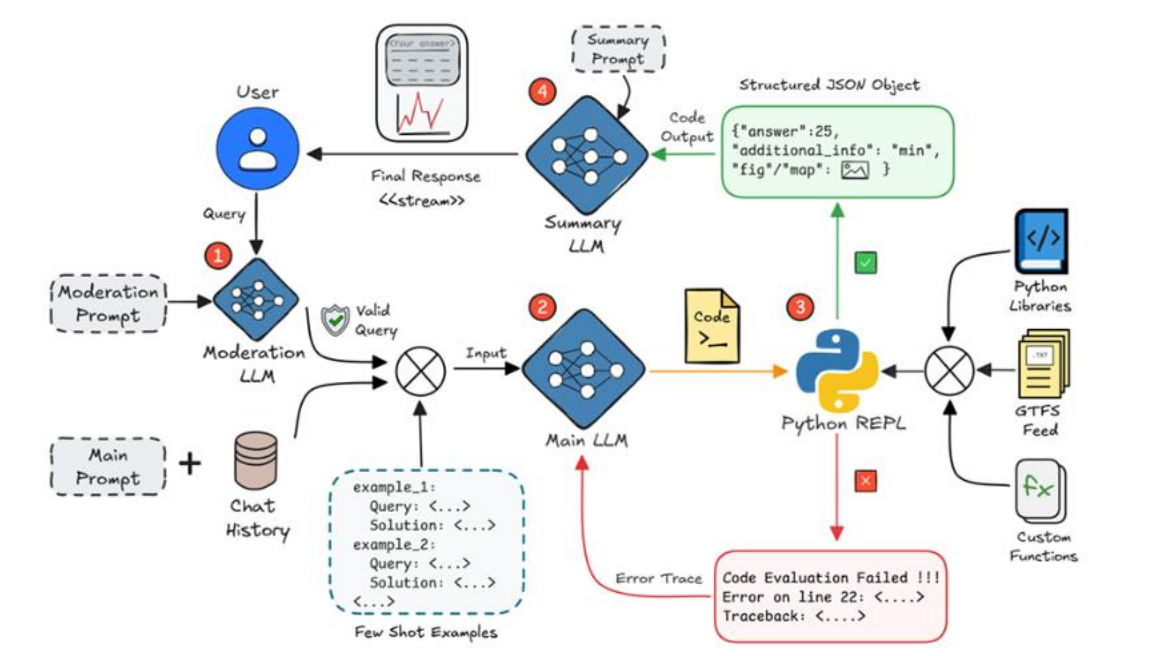


Figure 1: Conceptual Framework [1]

Data Gathering

The dataset is sourced from General Transit Feed Specification (GTFS), an open data standard for sharing public transit schedules and geographic information. The dataset contains information regarding routes, calendar, routes, shapes, stop_times, stops, and trips

centered around Metro Manila. It will serve as the main source of data for building and evaluating the model. Table 1 summarizes the primary GTFS files.

GTFS File	Content
stops.txt	Defines physical stop locations with stop_id, stop_name, stop_lat, stop_lon, and optional fields including wheelchair_boarding.
routes.txt	Defines transit routes with route_id , route_short_name, route_long_name, and route_type (e.g., 0=tram, 3=bus, 1=metro).
trips.txt	Maps individual trips to routes and service calendars. Fields include trip_id , route_id, service_id, shape_id, direction_id, and trip_headsign.
stop_times.txt	Records scheduled arrival and departure times for each stop on each trip. Fields include trip_id, stop_id, arrival_time, departure_time, and stop_sequence.
calendar.txt	Defines recurring service patterns by day of week and a date range (start_date, end_date).
shapes.txt	Stores geographic polylines defining the physical path of a trip. Fields include shape_id, shape_pt_lat, shape_pt_lon, shape_pt_sequence, and shape_dist_traveled.

Table 1. Primary GTFS Files

Data Preparation

Data preparation in this study encompasses four sequential sub-processes: (1) GTFS Feed Validation and Database Setup, (2) Field Normalization and Data Cleaning, (3) M-Schema Generation, and (4) Embedding and Vector Indexing.

1. GTFS Database Setup

- Description: All .txt files are ingested into a PostgreSQL database with a custom schema that mirrors the GTFS relational model. Primary keys, foreign keys, and appropriate data types are enforced to maintain referential integrity.
- Output: A relational PostgreSQL database containing validated and fully ingested GTFS data

2. Field Normalization and Data Cleaning

- Description: Time fields are converted to integer seconds; date fields are cast to SQL DATE types, and geographic coordinates are stored as 'FLOAT'. Missing shape_dist_traveled values are pre-computed using geodesic distances, and unreferenced records are removed.
- Output: Cleaned and normalized dataset

3. M-Schema Generation

- Description: M-Schema documents encode the relational schema in a structured, model-friendly JSON format. The generation process involves:
 1. Extracting schema info via SQLAlchemy (table names, columns, types, keys).

2. Adding GTFS-specific semantic annotations for each column.
 3. Sampling representative rows as illustrative examples.
 4. Serializing the document as both JSON and natural language text for embedding.
- Output: M-Schema GTFS JSON

Model Setup

This section describes the models and technologies used in the development of the proposed retrieval-augmented framework for interacting with the GTFS data.

Embedding Model: The system uses a sentence transformer-based embedding model to convert the generated M-Schema GTFS JSON into dense vector representations. These embeddings capture the semantic meaning of the text and enable similarity search between submitted queries and stored knowledge base documents. This allows the system to retrieve relevant schema information even when the query wording differs from the stored data.

Large Language Models: Large language models (LLMs) are used to interpret user queries and generate natural language responses. The LLM receives the user query together with retrieved schema documents from the vector database and generates responses based on the provided contextual information. This enables the system to understand natural language queries and map them to structured transit data.

Vector Database: A vector database stores the embeddings generated from schema documents. When a user query is submitted, it is converted into an embedding and

compared with stored vectors using similarity search. The database then retrieves the most relevant schema documents to provide context for the language model.

Retrieval-Augmented Generation: The framework follows a Retrieval-Augmented Generation (RAG) architecture. In this approach, the system first retrieves relevant schema documents from the vector database and then provides them as context to the language model. The language model uses this retrieved information to generate responses related to transit data queries.

Experimental Setup

This section describes how the experiment is conducted to evaluate the proposed framework.

Experimental Environment

The system is implemented in Python and executed within Google Colab, using libraries for machine learning, natural language processing, and database management.

Dataset

The experiment uses a GTFS dataset, which is imported into a relational database. The tables are then converted into JSON using the M-Schema framework. Schema descriptions—including table definitions, relationships, and sample records—are extracted from these JSON documents to build the retrieval knowledge base.

Retrieval Corpus

The JSON documents are split into text segments, converted into vector embeddings, and stored in the vector database with metadata. This collection forms the knowledge base for query retrieval.

Query Processing

Submitted queries are embedded using the same model and matched against the knowledge base via similarity search. Retrieved documents are combined with the query and fed to the language model, which generates the response.

Model Framework

The proposed framework utilizes a Retrieval-Augmented Generation (RAG) pipeline to ground the Large Language Model (LLM) in the GTFS data, ensuring that the generated responses strictly adhere to the retrieved context. By integrating a retriever that performs semantic similarity searches over a vector database, the framework provides the generator with explicit external knowledge rather than relying solely on the model's internal parameters. The addition of the RAG pipeline improves the performance of the LLM by reducing hallucinations and minimizing the inclusion of unrelated information.

To address the inherent structural challenges of GTFS data—specifically that its information is fragmented across multiple interrelated text files—the framework incorporates the M-Schema framework to construct a unified representation. M-Schema generates a semi-structured JSON document that consolidates these disparate files into a single, model-friendly format. This representation minimizes fragmentation while preserving essential meanings and the complex relational structures of the transit data,

providing the LLM with a better roadmap of the GTFS data for more accurate query synthesis and reasoning. Figure 2 shows the overall proposed framework.

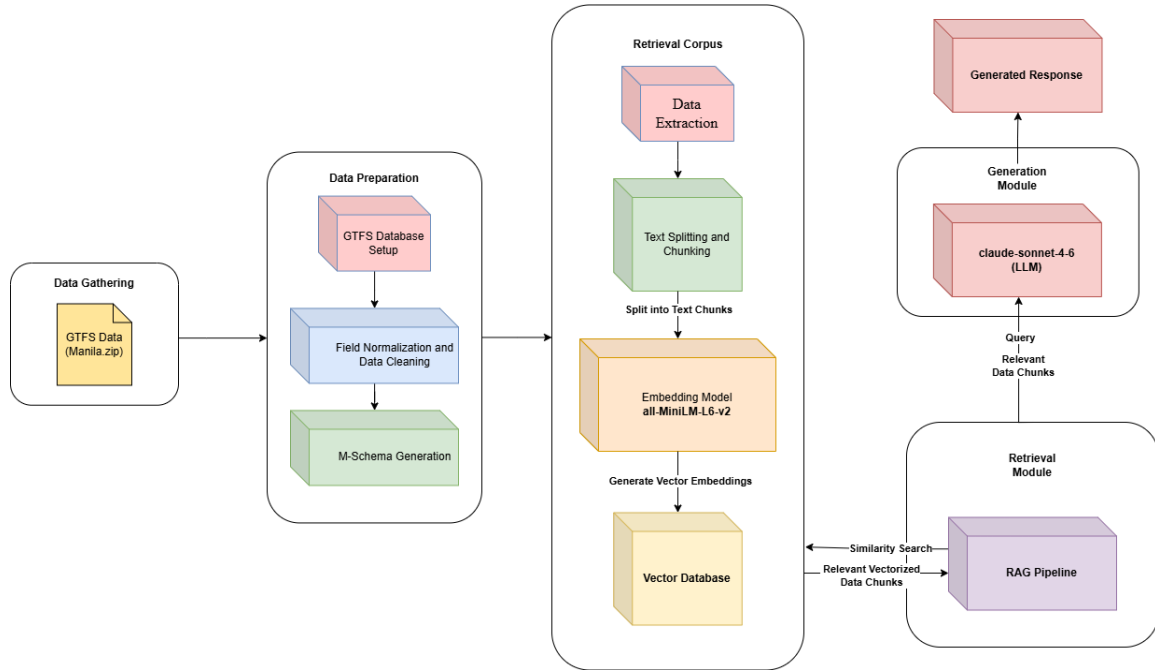


Figure 2. Proposed Conceptual Framework

Model Evaluation

The model is to be evaluated to assess its ability to accurately retrieve and generate relevant information from GTFS data.

1. **Performance Metrics:** The framework will be evaluated using F1 score, accuracy, precision, and recall measuring the effectiveness and quality of the retrieved GTFS data.
2. **Gap Analysis:** Comparison with TransitGPT will identify differences in context recall and retrieval accuracy, especially for queries requiring integration across multiple GTFS tables.

Chapter 4

RESULTS AND DISCUSSION

This chapter presents the results of the empirical evaluation conducted to assess the performance of the proposed GTFS-RAG framework and to compare it against the existing TransitGPT framework. The evaluation is organized around four core research questions that guided this study: how raw GTFS data can be transformed into a structured relational database with M-Schema representations, how an integrated RAG pipeline can be developed to process transit queries, how effective the proposed system is in terms of standard machine learning metrics, and how it performs relative to a runtime code execution baseline.

4.1 Benchmark Setup and Evaluation Protocol

The evaluation was conducted on a standardized benchmark of 75 questions derived from the Manila General Transit Feed Specification (GTFS) dataset. Both systems were evaluated using the same benchmark under identical dataset conditions and systems were run using Claude Sonnet 4.6 as the underlying large language model. Performance is reported using accuracy, precision, recall, and F1 score, derived from a per-question confusion matrix assignment following the procedures described in Chapter 3.

The questions are stratified across three difficulty levels: 25 Easy questions, 25 Medium questions, and 25 Hard questions. Easy questions involve straightforward single-

table operations such as direct counts and simple attribute lookups. Medium questions introduce filtering constraints, basic joins, and calendar-aware queries. Hard questions require multi-table joins, cross-agency reasoning, and frequency-style computations that span several interdependent GTFS files. This stratified design was deliberately constructed to expose architectural differences between systems at varying levels of relational complexity.

Each question is associated with a ground truth answer computed by executing verifiable queries against the Manila GTFS dataset, ensuring that gold labels are factually grounded. Each predicted answer is evaluated through a normalized exact-match procedure against its corresponding ground truth. The evaluation framework assigns each question to one of four confusion categories: True Positive (TP) when the answer is answerable and the prediction matches the ground truth; True Negative (TN) when both the ground truth and prediction indicate unanswerability; False Positive (FP) when the model commits to an incorrect answer; and False Negative (FN) when the model abstains on an answerable question.

In the current benchmark, all 75 questions are answerable. Consequently, no TN outcomes were recorded for either system in this evaluation run. The aggregate confusion distribution is reported alongside the headline metrics to enable transparent interpretation of results. In particular, when FN equals zero and TN equals zero, recall equals 1.0 and accuracy equals precision, as both are computed as TP over 75. This mathematical consequence of the benchmark structure is noted explicitly in the results tables to prevent misinterpretation.

The benchmark questions are further annotated by category type — count, filter_count, lookup, aggregation, multi_table_count, and multi_table_aggregation — and by whether the question requires access to multiple GTFS tables simultaneously. These annotations provide the granularity necessary for a deeper analysis of where each system succeeds or fails beyond the headline accuracy figure.

4.1.1 Benchmark Questions

The complete set of 75 benchmark questions is presented in Table 2, organized by difficulty tier. Each entry includes the question identifier, the natural language question text, and the assigned question category. These questions span a range of relational complexity and were constructed to systematically expose the architectural strengths and limitations of each evaluated system.

Difficulty: Easy (25 Questions)

ID	Question	Category
q001	How many routes are there in the Manila GTFS feed?	count
q002	How many stops are there in the Manila GTFS feed?	count
q003	How many trips are there in the Manila GTFS feed?	count
q004	How many bus routes (route_type = 3) are in the Manila feed?	filter_count
q005	How many rail routes (route_type = 2) are in the Manila feed?	filter_count
q006	How many transit agencies are in the Manila GTFS feed?	count
q007	How many stops have 'LRT' in their name?	filter_count
q008	How many stops have 'MRT' in their name?	filter_count

q009	How many unique service IDs are defined in the calendar?	count
q010	How many unique shape IDs are in the shapes file?	count
q011	How many frequency records are in the frequencies file?	count
q012	How many routes does LTFRB operate?	filter_count
q013	How many routes does LRTA operate?	filter_count
q014	How many routes does MRTC operate?	filter_count
q015	How many routes does PNR operate?	filter_count
q016	How many routes does The Fort Bus operate?	filter_count
q017	What is the name of the northernmost stop in the Manila feed?	lookup
q018	What is the name of the southernmost stop in the Manila feed?	lookup
q019	What is the name of the easternmost stop in the Manila feed?	lookup
q020	What is the name of the westernmost stop in the Manila feed?	lookup
q021	How many stops contain the word 'Station' in their name?	filter_count
q022	How many stops contain the word 'Terminal' in their name?	filter_count
q023	What are the two distinct route types available in the Manila feed?	lookup
q024	What is the full name of the agency with agency_id LRTA?	lookup
q025	What is the full name of the agency with agency_id MRTC?	lookup

Difficulty: Medium (25 Questions)

ID	Question	Category
q026	Which transit agency operates the most routes in Manila?	aggregation
q027	How many trips operate on weekdays (Monday to Friday)?	multi_table_count
q028	How many trips operate on weekends (Saturday or Sunday)?	multi_table_count
q029	How many trips operate on Saturdays?	multi_table_count
q030	How many trips operate on Sundays?	multi_table_count
q031	Which route has the most number of trips scheduled?	multi_table_aggregation
q032	How many routes have more than one trip scheduled?	multi_table_count
q033	How many routes have exactly one trip scheduled?	multi_table_count
q034	How many total stop_times records are in the feed?	count
q035	How many unique trips appear in the stop_times table?	count
q036	How many unique stops appear in the stop_times table?	count
q037	What is the maximum stop_sequence value across all trips?	aggregation
q038	How many unique service IDs are actually used in the trips table?	count
q039	How many trips have an assigned shape_id?	filter_count
q040	How many unique shape IDs are referenced in the trips table?	count
q041	How many trips have no headsign defined?	filter_count
q042	How many routes have no long name defined?	filter_count
q043	What is the average latitude of all stops in the Manila feed?	aggregation
q044	How many LRT 1 trips are scheduled in total?	multi_table_count

q045	What is the most frequently appearing stop in the stop_times table?	aggregation
q046	How many LTFRB routes have more than one trip scheduled?	multi_table_count
q047	How many routes have trips with no shape assigned?	multi_table_count
q048	What is the maximum stop_sequence for any LRT 1 trip?	multi_table_aggregation
q049	How many routes does LRTA operate in the Manila feed?	filter_count
q050	What is the average number of trips per route across all agencies?	multi_table_aggregation

Difficulty: Hard (25 Questions)

ID	Question	Category
q051	How many unique stops does LRT 1 serve?	multi_table_count
q052	How many unique stops does MRT-3 serve?	multi_table_count
q053	How many unique stops does LRT 2 serve?	multi_table_count
q054	How many unique stops does PNR MC serve?	multi_table_count
q055	What is the average number of stops per trip for bus routes?	multi_table_aggregation
q056	What is the average number of stops per trip for rail routes?	multi_table_aggregation
q057	How many routes serve the Baclaran LRT stop?	multi_table_count
q058	How many routes serve the EDSA LRT stop?	multi_table_count
q059	How many unique stops are served on weekdays?	multi_table_count
q060	How many unique stops are served on weekends?	multi_table_count
q061	How many unique stops do LRTA routes serve in total?	multi_table_count
q062	How many unique stops do LTFRB routes serve in total?	multi_table_count

q063	Which stop is served by the most number of routes?	multi_table_aggregation
q064	How many routes serve the stop named Taft Ave, Manila?	multi_table_count
q065	How many bus trips serve more than 50 stops?	multi_table_count
q066	How many bus trips serve fewer than 10 stops?	multi_table_count
q067	How many stop_time records belong to weekday trips?	multi_table_count
q068	How many stop_time records belong to rail routes?	multi_table_count
q069	Do LRT 1 and MRT-3 share any stops in common?	multi_table_count
q070	How many routes operate on both weekdays and weekends?	multi_table_count
q071	How many LTFRB routes have exactly one trip?	multi_table_count
q072	How many routes have trips that appear in stop_times?	multi_table_count
q073	How many routes have no shape assigned to any of their trips?	multi_table_count
q074	What is the total number of stop_time records for all LRT routes combined?	multi_table_count
q075	How many unique routes serve stops that contain 'LRT' in their name?	multi_table_count

Table 2. Complete Benchmark Questions by Difficulty Level (n = 75)

4.2 Results of the TransitGPT Framework

4.2.1 Performance Metrics

The TransitGPT framework achieved an overall accuracy of 0.4133, a precision of 0.4133, a recall of 1.0000, and an F1 score of 0.5849. These aggregate performance metrics are presented in Table 3.

Metric	Score
Accuracy	0.4133
Precision	0.4133
Recall	1.0000
F1 Score	0.5849

Table 3. Performance Metrics — TransitGPT Framework (claude-sonnet-4-6, n=75)

The accuracy of 0.4133 indicates that TransitGPT answered 31 out of 75 benchmark questions correctly. The precision value of 0.4133 is numerically equal to accuracy in this evaluation, a consequence of the system's zero-FN behavior since the pipeline always commits to a concrete answer and never abstains, the total number of positive predictions equals all 75 questions. The recall of 1.0000 is the most distinctive metric for TransitGPT. It reflects the fact that every answerable question received an attempted answer, meaning the system never declined to respond regardless of question complexity. This perfect recall is a direct consequence of the program synthesis architecture, in which the Python REPL executes generated code and the error-correction loop continues until a concrete output is produced. The F1 score of 0.5849, representing the harmonic mean of precision and recall, reflects the overall balance of the system's commitment strategy.

4.2.2 Confusion Matrix Analysis

The confusion matrix for TransitGPT is presented in Table 4. Of the 75 answerable benchmark questions, the system recorded 31 True Positives, 0 True Negatives, 44 False Positives, and 0 False Negatives.

	Predicted Positive	Predicted Negative
Actual Positive	31 True Positive	0 False Negative
Actual Negative	44 False Positive	0 True Negative

Table 4. Confusion Matrix — TransitGPT Framework (claude-sonnet-4-6, n=75)

Based on the table, it can be observed that the FN is equal to zero. The program synthesis pipeline is architecturally designed to always return a result: the Main LLM generates Python code, the Python REPL executes it against the GTFS DataFrames, and if an error occurs, the error trace is returned to the model for code refinement. This iterative loop terminates only when a valid output is produced, which eliminates abstention as a response mode. As a result, all 75 questions received either a correct answer (TP = 31) or an incorrect one (FP = 44). The zero-FN property directly elevates recall to 1.0 and causes precision to equal accuracy, since every prediction is a positive commitment.

The FP count of 44 reveals that while TransitGPT always attempted an answer, it produced an incorrect result on more than half of the questions it failed to answer correctly.

These false positives arise from two primary sources: logical errors in the generated Python code that produce numerically incorrect outputs despite successful execution, and exact-match failures where the computed answer is conceptually correct but lexically mismatched with the gold label. The TN count of zero is a benchmark-level property, not an architectural one since all 75 questions are answerable, there were no unanswerable queries for the system to correctly identify.

4.2.3 Performance by Difficulty Level

TransitGPT's difficulty-level breakdown reveals a performance profile that improves markedly at the Hard tier. On Easy questions, the system answered 11 out of 25 correctly, yielding an accuracy and precision of 0.4400 and an F1 score of 0.6111. On Medium questions, it answered 7 out of 25 correctly, with accuracy and precision of 0.2800 and an F1 score of 0.4375. On Hard questions, TransitGPT answered 13 out of 25 correctly, achieving accuracy and precision of 0.5200 and an F1 score of 0.6842 — the highest tier-level F1 score of any difficulty level for either system.

Difficulty	Correct	Total	TP	TN	FP	FN	Accuracy	Precision	F1
Easy	11	25	11	0	14	0	0.4400	0.4400	0.6111
Medium	7	25	7	0	18	0	0.2800	0.2800	0.4375
Hard	13	25	13	0	12	0	0.5200	0.5200	0.6842
Total	31	75	31	0	44	0	0.4133	0.4133	0.5849

Table 5. TransitGPT Performance by Difficulty Level

The Hard difficulty tier is where the most significant performance advantage of TransitGPT manifests relative to the GTFS-RAG system. TransitGPT's ability to generate and execute Python code that joins multiple GTFS DataFrames, applies temporal constraints, and computes aggregates programmatically allows it to address the class of questions that the GTFS-RAG system fails to resolve. Recall remained at 1.0000 across all three difficulty tiers because the zero-FN property of the program synthesis architecture is unconditional and it does not degrade with increasing question complexity.

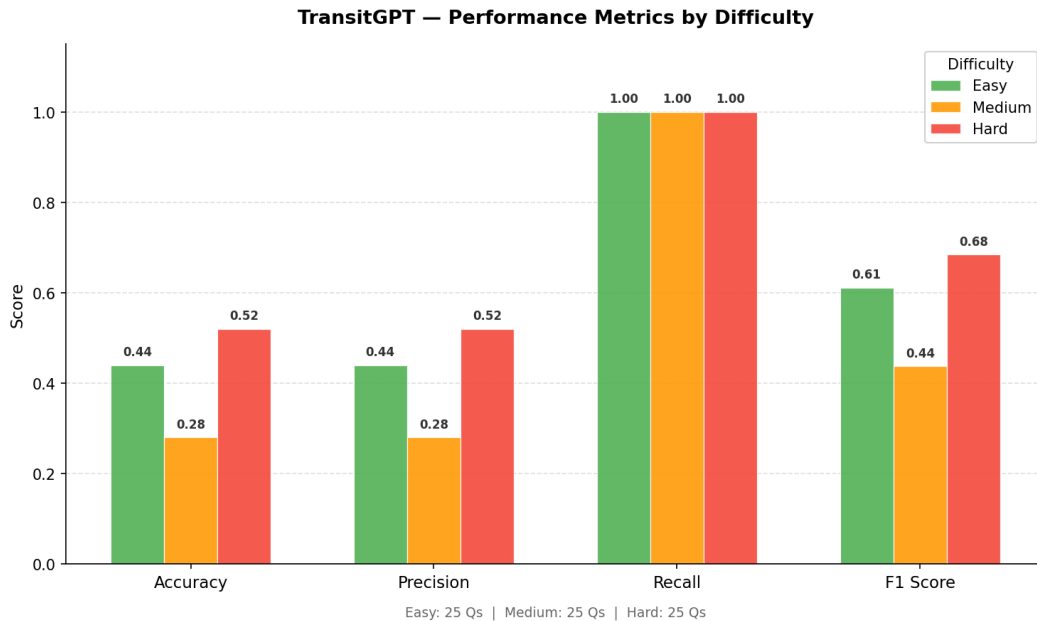


Figure 3. TransitGPT Performance Metrics by Difficulty Level

4.2.4 Performance by Question Category

By question category, TransitGPT demonstrated its most decisive strength in the `multi_table_count` category, answering 17 out of 31 questions correctly — a success rate of 54.8%. On `filter_count` questions, TransitGPT answered 9 out of 15 correctly (60.0%). These two categories collectively account for 46 of the 75 benchmark questions and

represent the domains in which program synthesis provides the most significant advantage over retrieval-based generation.

Conversely, TransitGPT exhibited notable weaknesses in categories that require direct entity matching rather than relational computation. On count questions, TransitGPT answered only 5 out of 12 correctly (41.7%). On lookup questions, TransitGPT answered 0 out of 7 correctly (0.0%), and on aggregation questions, it answered 0 out of 4 correctly (0.0%). These zero-performance categories suggest that exact entity name matching in generated code is susceptible to mismatches when queries use informal or abbreviated names that differ from stored GTFS identifiers.

TransitGPT — Correct / Wrong per Question								
Easy 11/25 correct			Medium 7/25 correct			Hard 13/25 correct		
q001 X	q002 X	q003 ✓	q026 X	q027 ✓	q028 ✓	q051 ✓	q052 ✓	q053 ✓
q004 X	q005 ✓	q006 ✓	q029 ✓	q030 ✓	q031 X	q054 ✓	q055 X	q056 X
q007 ✓	q008 ✓	q009 X	q032 X	q033 X	q034 ✓	q057 X	q058 X	q059 ✓
q010 ✓	q011 ✓	q012 ✓	q035 X	q036 X	q037 X	q060 ✓	q061 ✓	q062 ✓
q013 ✓	q014 X	q015 ✓	q038 X	q039 X	q040 X	q063 X	q064 X	q065 X
q016 X	q017 X	q018 X	q041 ✓	q042 X	q043 X	q066 X	q067 ✓	q068 X
q019 X	q020 X	q021 X	q044 X	q045 X	q046 X	q069 X	q070 ✓	q071 X
q022 ✓	q023 X	q024 X	q047 X	q048 X	q049 ✓	q072 ✓	q073 X	q074 X
q025 X			q050 X			q075 ✓		

Figure 4. TransitGPT Per-Question Correctness Grid Across All 75 Benchmark Items

4.2.5 Multi-Table vs. Single-Table Performance

TransitGPT answered 17 out of 37 multi-table questions correctly (45.9%) and 14 out of 38 single-table questions correctly (36.8%). The higher accuracy on multi-table questions reflects the program synthesis architecture's core strength: generated code can join multiple DataFrames simultaneously, making complex relational operations no more architecturally challenging than simple ones. The moderate single-table accuracy of 36.8%

indicates that even for simpler questions, the code generation pipeline does not guarantee correctness — logical errors and exact-match failures can occur on any question type.

4.3 Results of the Proposed GTFS-RAG Framework

4.3.1 Performance Metrics

The GTFS-RAG system achieved an overall accuracy of 0.2400, a precision of 0.3103, a recall of 0.5143, and an F1 score of 0.3871. These aggregate performance metrics are presented in Table 6.

Metric	Score
Accuracy	0.2400
Precision	0.3103
Recall	0.5143
F1 Score	0.3871

Table 6. Performance Metrics — GTFS-RAG Framework (claude-sonnet-4-6, n=75)

The accuracy of 0.2400 indicates that the GTFS-RAG system correctly answered 18 out of 75 benchmark questions, representing a 17.33 percentage-point deficit relative to TransitGPT. The precision of 0.3103 indicates that slightly more than three in ten of the system's committed predictions were correct, that is, among the 58 questions to which the system provided a definitive answer rather than abstaining, only 18 were correct. This precision figure reflects both the retrieval pipeline's tendency to surface incomplete or only partially relevant schema context and the language model's subsequent generation of plausible but incorrect answers from that context.

The recall of 0.5143 is a compound measure in this evaluation context. It reflects that the system correctly answered approximately half of the questions it chose to attempt (18 correct out of 35 non-abstaining responses), but it is simultaneously depressed by the 17 False Negative cases in which the system abstained from answering entirely. Because recall is computed as TP divided by the total number of answerable positives ($TP + FN = 18 + 17 = 35$ attempted, with 17 abstentions), the abstention behavior directly reduces recall relative to TransitGPT's zero-FN result. The F1 score of 0.3871 reflects the harmonic mean of precision and recall and represents the overall quality balance of the system — meaningfully lower than TransitGPT's 0.5849 but indicating that the pipeline is not entirely non-functional.

4.3.2 Confusion Matrix Analysis

The confusion matrix for the GTFS-RAG system is presented in Table 7. Of the 75 answerable benchmark questions, the system recorded 18 True Positives, 0 True Negatives, 40 False Positives, and 17 False Negatives.

	Predicted Positive	Predicted Negative
Actual Positive	18 True Positive	17 False Negative
Actual Negative	40 False Positive	0 True Negative

Table 7. Confusion Matrix — GTFS-RAG Framework (claude-sonnet-4-6, $n=75$)

The confusion matrix reveals two structurally distinct failure modes operating simultaneously within the GTFS-RAG pipeline. The first failure mode, represented by the 40 False Positive cases, occurs when the system commits to an answer that does not match the ground truth. In these instances, the retrieval mechanism successfully surfaced schema context that was plausible and semantically related to the query, but the retrieved information was either incomplete, missing a critical foreign key or table; or imprecise, leading the language model to generate a response that failed the normalized exact-match criterion. The FP count of 40 is the dominant error category and is primarily concentrated in the Medium and Hard difficulty tiers, where multi-table context is essential for a correct answer.

The second failure mode, represented by the 17 False Negative cases, occurs when the system declines to answer entirely, typically by outputting UNKNOWN or an equivalent abstention marker. This behavior is most prevalent in the Hard difficulty tier, where 10 of the 17 FNs are concentrated. The FN cases represent instances in which the retrieval pipeline failed to return context that was sufficient for the language model to form a confident response. Paradoxically, this abstention behavior — while reducing recall — may represent a form of epistemic calibration absent from TransitGPT: the system recognizes the inadequacy of its retrieved context and declines to fabricate an answer, which in high-stakes transit information applications could be preferable to a committed but incorrect response. The TN count of zero is a benchmark-level property shared by both systems, reflecting that all 75 questions in the current benchmark are answerable.

4.3.3 Performance by Difficulty Level

The GTFS-RAG system's performance differs substantially across difficulty levels. On Easy questions, the system answered 9 out of 25 correctly, recording 9 TPs, 11 FPs, and 5 FNs, yielding an accuracy of 0.3600, precision of 0.4500, recall of 0.6429, and F1 score of 0.5294. On Medium questions, 7 out of 25 were answered correctly, with 7 TPs, 16 FPs, and 2 FNs, yielding accuracy of 0.2800, precision of 0.3043, recall of 0.7778, and F1 score of 0.4375. The most pronounced decline occurred in the Hard category, where only 2 out of 25 questions were answered correctly, resulting in 2 TPs, 13 FPs, and 10 FNs, with accuracy of 0.0800, precision of 0.1333, recall of 0.1667, and F1 score of 0.1481.

Difficulty	Correct	Total	TP	TN	FP	FN	Accuracy	Precision	F1
Easy	9	25	9	0	11	5	0.3600	0.4500	0.5294
Medium	7	25	7	0	16	2	0.2800	0.3043	0.4375
Hard	2	25	2	0	13	10	0.0800	0.1333	0.1481
Total	18	75	18	0	40	17	0.2400	0.3103	0.3871

Table 8. GTFS-RAG Performance by Difficulty Level

While the system demonstrated limited but measurable capability on simpler queries, its effectiveness deteriorates significantly as relational complexity increases. The Hard questions expose the fundamental limitation of the retrieval-based architecture: the inability to programmatically compose relational operations across the full dataset. The 10 FNs observed in the Hard category indicate that the system frequently recognized its inability to provide a confident answer and defaulted to abstention, rather than

hallucinating an incorrect response. This contrasts with TransitGPT, which recorded 13 correct answers on Hard questions with zero abstentions.

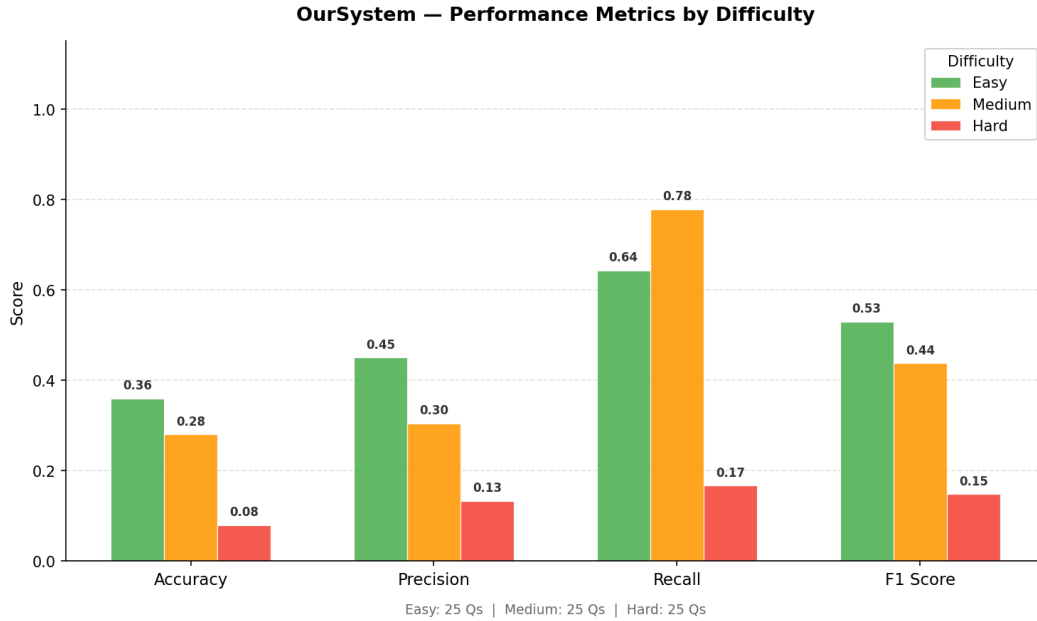


Figure 5. GTFS-RAG System Performance Metrics by Difficulty Level

4.3.4 Performance by Question Category

A breakdown by question category provides additional insight into the nature of the GTFS-RAG system's performance characteristics. The system performed best on count-type questions (8 correct out of 12, 66.7%), which involve straightforward enumerations of single entities and typically require access to only one GTFS table. Performance declined on filter_count questions (3 out of 15, 20.0%), which introduce conditional filtering. The most significant failure occurred in the multi_table_count category, where the system answered only 2 out of 31 questions correctly — a success rate of 6.5%. On multi_table_aggregation (1 out of 6), lookup (3 out of 7), and aggregation (1 out of 4) categories, performance was similarly limited.

These category-level results demonstrate that the GTFS-RAG system is most effective when queries can be resolved using a narrow subset of schema information from a single table and least effective when queries require integrating information from multiple tables simultaneously. This pattern is consistent with the architectural properties of the retrieval pipeline described in Chapter 3, where top-k similarity retrieval selects schema chunks and the language model infers answers without executing any relational operations against the live database.

OurSystem — Correct / Wrong per Question								
Easy 9/25 correct			Medium 7/25 correct			Hard 2/25 correct		
q001 ✓	q002 ✗	q003 ✓	q026 ✓	q027 ✗	q028 ✗	q051 ✓	q052 ✗	q053 ✓
q004 ✗	q005 ✗	q006 ✓	q029 ✗	q030 ✗	q031 ✗	q054 ✗	q055 ✗	q056 ✗
q007 ✗	q008 ✗	q009 ✓	q032 ✗	q033 ✗	q034 ✓	q057 ✗	q058 ✗	q059 ✗
q010 ✗	q011 ✓	q012 ✗	q035 ✓	q036 ✗	q037 ✗	q060 ✗	q061 ✗	q062 ✗
q013 ✓	q014 ✗	q015 ✗	q038 ✓	q039 ✗	q040 ✗	q063 ✗	q064 ✗	q065 ✗
q016 ✗	q017 ✗	q018 ✗	q041 ✓	q042 ✗	q043 ✗	q066 ✗	q067 ✗	q068 ✗
q019 ✗	q020 ✗	q021 ✗	q044 ✗	q045 ✗	q046 ✗	q069 ✗	q070 ✗	q071 ✗
q022 ✗	q023 ✓	q024 ✓	q047 ✗	q048 ✓	q049 ✓	q072 ✗	q073 ✗	q074 ✗
q025 ✓			q050 ✗			q075 ✗		

Figure 6. GTFS-RAG Per-Question Correctness Grid Across All 75 Benchmark Items

4.3.5 Sample Evaluation Outputs

To illustrate the nature of the system's responses across difficulty tiers, representative sample questions drawn from the benchmark evaluation logs are presented below. These samples provide concrete examples of how the GTFS-RAG pipeline generates responses at each level of complexity.

```
[5/75] q005 [easy]: How many rail routes (route_type = 2) are in the Manila feed?
Ground truth : 4
Predicted    : 2 [WRONG]
Retrieved    : ['routes', 'routes', 'shapes'] sims=['0.548', '0.532', '0.418']
Time        : 2.652s

[6/75] q006 [easy]: How many transit agencies are in the Manila GTFS feed?
Ground truth : 5
Predicted    : 5 [OK]
Retrieved    : ['agency', 'routes', 'routes'] sims=['0.472', '0.472', '0.437']
Time        : 2.244s

[7/75] q007 [easy]: How many stops have 'LRT' in their name?
Ground truth : 36
Predicted    : I need to count stops with 'LRT' in their name from the stops table.

From the schema, I can see stops like "Baclaran LRT", "EDSA LRT", "Libertad LRT", etc. with LTFRB_ IDs, plus LRT station stops with their own IDs.

Based on the GTFS data for Manila, counting all stops where stop_name contains 'LRT':

- LRT 1 has 20 stations (both directions represented)
- LRT 2 has 13 stations
- Plus LTFRB bus stop references to LRT stations

Let me reason from the provided data. The stops table has 4857 total rows. The sample shows LRT-named stops with LTFRB_ prefix IDs.

From the route stops listed:
- LRT 1: Baclaran LRT, EDSA LRT, Libertad LRT, Gil Puyat LRT, Vito Cruz LRT, Quirino LRT, Pedro Gil LRT, United Nations LRT, Central Terminal LRT, Carriedo LRT, Doroteo Jose LRT, Bambang LRT, Tayuman LRT, Blumentritt LRT, Abad Santos LRT, R. Papa LRT, 5th Ave LRT, Monumento LRT, Balintawak LRT, Roosevelt LRT = 20 stops
- LRT 2: Santolan LRT, Katipunan LRT, Anonas LRT, V. Mapa LRT, Betty Go Belmonte LRT, Cubao LRT, J. Ruiz LRT, Gilmore LRT, Legarda LRT, Pureza LRT, Recto LRT = additional stops

**33** [WRONG]
Retrieved    : ['frequencies', 'stops', 'stop_times'] sims=['0.461', '0.413', '0.403']
Time        : 8.413s
```

Figure 7. Sample GTFS-RAG Responses — Easy Difficulty Questions

```
[26/75] q026 [medium]: Which transit agency operates the most routes in Manila?
Ground truth : LTFRB
Predicted    : LTFRB [OK]
Retrieved    : ['routes', 'routes', 'agency'] sims=['0.511', '0.502', '0.496']
Time        : 2.323s

[27/75] q027 [medium]: How many trips operate on weekdays (Monday to Friday)?
Ground truth : 1840
Predicted    : I need to find trips that operate on weekdays (Monday to Friday). This means finding service_ids where monday=1, tuesday=1, wednesday=1, thursday=1, friday=1, then counting trips with those service_ids.

From the calendar data:
- service_id=882002: M:1, T:1, W:1, Th:1, F:1, Sa:1, Su:0 (operates weekdays)
- service_id=880772: M:1, T:1, W:1, Th:1, F:1, Sa:0, Su:0 (operates weekdays only)
- service_id=724594: M:1, T:1, W:1, Th:1, F:1, Sa:1, Su:1 (operates weekdays)
- service_id=882006: Su only - NO
- service_id=881444: Su only - NO
- service_id=881445: Su only - NO

So weekday-operating service_ids include: 724594, 882002, 880772 (and potentially others in the full 12-row calendar table).

The question asks how many trips operate on weekdays. I need to count trips whose service_id has monday through friday = 1.

Based on the data provided, I can identify that service_ids 724594, 882002, and 880772 operate on weekdays. The trips table has 1862 total rows, and I need to count those with these service_ids.

From the context, the trips with service_id=880772 and 882002 and 724594 would be the weekday trips. Without the exact query result, I'll work with what's available.

Given the total trips = 1862 and the structure, the weekday trips would be those associated with service_ids that have M-F = 1.

UNKNOWN [WRONG]
Retrieved    : ['calendar', 'trips', 'calendar'] sims=['0.574', '0.506', '0.419']
Time        : 8.807s

[28/75] q028 [medium]: How many trips operate on weekends (Saturday or Sunday)?
Ground truth : 1832
Predicted    : I need to find trips that operate on weekends (Saturday or Sunday) by joining trips with calendar data.

From the calendar data:
- service_id 882006: Sunday only (Su:1)
- service_id 882002: Mon-Sat (Sa:1, Su:0)
- service_id 880772: Mon-Fri only (Sa:0, Su:0)
- service_id 881444: Sunday only (Su:1)
- service_id 881445: Sunday only (Su:1)
```

Figure 8. Sample GTFS-RAG Responses — Medium Difficulty Questions

```

[51/75] q051 [hard]: How many unique stops does LRT 1 serve?
Ground truth : 20
Predicted    : 20 [OK]
Retrieved    : ['frequencies', 'stop_times', 'stops'] sims=['0.416', '0.395', '0.387']
Time        : 1.999s

[52/75] q052 [hard]: How many unique stops does MRT-3 serve?
Ground truth : 13
Predicted    : UNKNOWN [WRONG]
Retrieved    : ['stop_times', 'stops', 'frequencies'] sims=['0.371', '0.339', '0.338']
Time        : 1.59s

[53/75] q053 [hard]: How many unique stops does LRT 2 serve?
Ground truth : 11
Predicted    : 11 [OK]
Retrieved    : ['stop_times', 'frequencies', 'stops'] sims=['0.411', '0.410', '0.399']
Time        : 1.857s

[54/75] q054 [hard]: How many unique stops does PNR MC serve?
Ground truth : 22
Predicted    : UNKNOWN [WRONG]
Retrieved    : ['stops', 'stop_times', 'frequencies'] sims=['0.326', '0.308', '0.302']
Time        : 1.805s

[55/75] q055 [hard]: What is the average number of stops per trip for bus routes?
Ground truth : 43.4
Predicted    : I need to calculate the average number of stops per trip for bus routes (LFRB routes).

From the GTFS data, bus routes are operated by LFRB. The stop_times table contains all stop sequences for all trips.

Based on the data provided:
- Total rows in stop_times: 79,396
- The LRT routes (LRT 1 and LRT 2) have known stop counts

For LRT 1: ~20 stops (Roosevelt to Baclaran, ~20 stations)
For LRT 2: ~13 stops (Santolan to Recto, ~13 stations)

LRT trips would account for a portion of the 79,396 rows. The bulk of trips are LFRB bus routes.

From the sample data, trip 880815 appears to be an LFRB trip (stop_id: LFRB_4977).

Looking at the frequencies table: 1,862 frequency entries, and the stop_times has 79,396 rows total.

To estimate bus stops per trip: The LFRB routes use stop IDs with "LFRB_" prefix. LRT stops also use LFRB_ prefix based on the sample data shown (LFRB_4944 = Baclaran

```

Figure 9. Sample GTFS-RAG Responses — Hard Difficulty Questions

4.3.6 Multi-Table vs. Single-Table Performance

Of the 37 questions flagged as requiring access to multiple tables, the GTFS-RAG system answered only 3 correctly (8.1%). In contrast, of the 38 single-table questions, the system answered 15 correctly (39.5%). This disparity of approximately 31 percentage points between multi-table and single-table accuracy is the most revealing diagnostic produced by the evaluation. It confirms that the system's retrieval mechanism, while partially effective for simple lookups, is fundamentally insufficient for the class of relational queries that constitute the majority of the Hard difficulty tier.

4.4 Comparative Analysis Between the Two Systems

Table 9 presents a side-by-side comparison of the aggregate metrics for both systems. TransitGPT outperforms GTFS-RAG on accuracy, precision, and F1 score. TransitGPT also holds a higher recall, though this comparison must be interpreted carefully: TransitGPT's perfect recall reflects its zero-FN behavior rather than a broad coverage of correct answers, while GTFS-RAG's recall is depressed by 17 abstention cases.

Metric	TransitGPT	GTFS-RAG	Difference (TGPT – RAG)
Accuracy	0.4133	0.2400	+0.1733
Precision	0.4133	0.3103	+0.1030
Recall	1.0000	0.5143	+0.4857
F1 Score	0.5849	0.3871	+0.1978
TP	31	18	+13
TN	0	0	0
FP	44	40	+4 (TransitGPT higher)
FN	0	17	−17 (RAG higher)
Avg. Exec. Time (s)	9.91	4.65	+5.26 (TransitGPT slower)

Table 9. Side-by-Side Comparative Metrics: TransitGPT vs. GTFS-RAG (claude-sonnet-4-6, n=75)

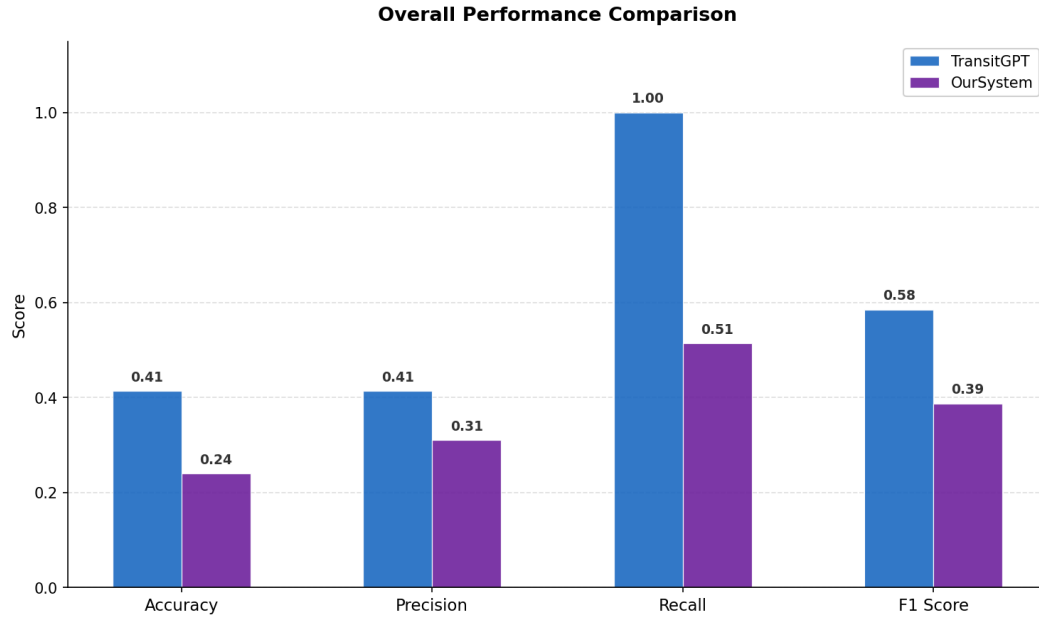


Figure 10. Overall Metric Comparison: TransitGPT vs. GTFS-RAG Across All Four Evaluation Metrics

At the difficulty-tier level, the comparison reveals a consistent pattern. On Easy questions, TransitGPT holds a moderate advantage (11 vs. 9 correct). On Medium questions, the two systems tie at 7 correct each, the one tier where the GTFS-RAG system achieves parity. On Hard questions, TransitGPT leads decisively: 13 correct versus only 2 for GTFS-RAG, a six-and-a-half-fold advantage. The Medium parity may reflect that some Medium questions are resolvable from a compact retrieval window. The Hard divergence is unambiguous and architecturally grounded.

Difficulty	TransitGPT Correct	GTFS-RAG Correct	Total Questions	Difference
Easy	11	9	25	+2 (TransitGPT)
Medium	7	7	25	0 (Tie)
Hard	13	2	25	+11 (TransitGPT)
Total	31	18	75	+13 (TransitGPT)

Table 10. Difficulty-Level Comparison: Correct Answers per System

At the category level, the comparison highlights complementary strengths. TransitGPT dominates on `multi_table_count` (17/31 vs. 2/31) and `filter_count` (9/15 vs. 3/15), while GTFS-RAG holds an advantage on `count` (8/12 vs. 5/12) and `lookup` (3/7 vs. 0/7).

Category	Total Qs	TransitGPT	GTFS-RAG	Advantage
count	12	5 (41.7%)	8 (66.7%)	GTFS-RAG
filter_count	15	9 (60.0%)	3 (20.0%)	TransitGPT
lookup	7	0 (0.0%)	3 (42.9%)	GTFS-RAG
aggregation	4	0 (0.0%)	1 (25.0%)	GTFS-RAG
multi_table_count	31	17 (54.8%)	2 (6.5%)	TransitGPT
multi_table_aggregation	6	0 (0.0%)	1 (16.7%)	GTFS-RAG

Table 11. Category-Level Performance Comparison: TransitGPT vs. GTFS-RAG

4.5 Retrieval Limitations in the RAG Architecture

The GTFS-RAG system's performance ceiling is, in significant part, a function of the retrieval mechanism's capacity to supply the language model with the precise contextual information necessary to answer each question. The framework employs the all-MiniLM-L6-v2 sentence embedding model to represent M-Schema chunks in a dense vector space, and retrieval is performed via cosine similarity search to identify the top-k most relevant schema segments. While this approach is effective for queries that can be answered by reasoning over a single schema chunk, it exhibits systematic limitations that explain the observed performance gaps.

The first limitation concerns retrieval coverage. When a question requires information from multiple GTFS tables — for instance, a query that must join trips, stop_times, and stops to count unique stops served by a particular rail line — the top-k retrieval window may not capture all requisite table schema chunks simultaneously. The analysis of multi-table performance (3 correct out of 37 multi-table questions) confirms this limitation empirically. The most frequently required tables in the benchmark are routes (needed by 42 questions), trips (42 questions), stop_times (28 questions), and stops (16 questions). Many Hard questions require all four of these tables to be present in the context window simultaneously, which the current top-k retrieval mechanism does not reliably achieve.

The second limitation involves context irrelevance. Cosine similarity between a query embedding and a schema chunk embedding reflects semantic relatedness but does not guarantee inferential sufficiency. A retrieved chunk may describe the correct table in

general terms while omitting the specific column or relational key required for a given question. In such cases, the language model receives plausible but incomplete context, leading either to an incorrect answer (FP) or to an abstention (FN). The 40 FP cases observed in the GTFS-RAG evaluation suggest that the model frequently attempted to answer despite incomplete context.

A third limitation arises from the general-purpose nature of the all-MiniLM-L6-v2 embedding model. This model was trained on broad natural language text and lacks domain-specific sensitivity to GTFS entities such as route type codes, agency identifiers, and service calendar date ranges. Finally, the current RAG pipeline does not incorporate a re-ranking mechanism or a query expansion strategy, the absence of which constitutes a primary direction for future improvement.

4.6 Relational Reasoning Challenges in GTFS Data

The General Transit Feed Specification imposes a highly normalized relational structure on transit data. Routes, trips, stop times, stops, and service calendars are stored in separate text files linked through primary and foreign key relationships. This normalization presents a fundamental challenge for retrieval-based language model frameworks: the information required to answer a single question is typically distributed across multiple tables, and no individual chunk from any single file contains a complete answer.

The M-Schema representation employed in this study partially addresses this challenge by encoding table metadata, column definitions, foreign key relationships, and representative sample rows into a structured JSON document. However, the M-Schema representation does not enable the model to execute relational queries; it provides

descriptive context about what the schema contains, not the results of joining tables on a given condition. A count question such as "How many routes are there in the Manila GTFS feed?" can be answered from the routes table schema chunk alone. In contrast, a multi_table_count question such as "How many unique stops does LRT 1 serve?" requires joining routes, trips, and stop_times — an operation that yields the observed 6.5% success rate on multi_table_count questions when attempted through retrieval alone.

The relational complexity of GTFS is further compounded by temporal constraints encoded in the calendar and calendar_dates tables, which govern service availability by day of week and date range. Several Hard benchmark questions require the model to respect these temporal constraints, filtering trips by their active service patterns before performing aggregations. For a retrieval-based system, reasoning correctly about these constraints without programmatic access to the calendar data is exceptionally difficult, and the observed FN and FP rates on Hard questions reflect this difficulty.

4.7 Advantages of Runtime Code Execution in TransitGPT

The program synthesis architecture of TransitGPT resolves several of the challenges described in Sections 4.5 and 4.6 by fundamentally changing the mechanism by which the language model interacts with GTFS data. Rather than answering from retrieved text, the model generates executable Python code that operates directly on pre-loaded pandas DataFrames. This approach confers three primary advantages that collectively account for TransitGPT's superior performance on multi-table and filter-intensive questions.

First, program synthesis provides complete relational access. The generated code can reference all GTFS tables simultaneously and apply standard DataFrame operations — merges, groupby, filtering, and value counting — across the full dataset. There is no retrieval ceiling: if the question requires joining four tables, the generated code can join four DataFrames. This is the direct explanation for TransitGPT's 45.9% success rate on multi-table questions versus GTFS-RAG's 8.1%.

Second, the error-correction loop enhances robustness. When generated code produces a runtime error, the error trace is fed back to the language model, which attempts to refine the code until a valid output is produced. This iterative self-correction mechanism allows the system to recover from syntactic errors, incorrect column name references, and logical mistakes in the initial code generation. The result is that the system consistently commits to a concrete answer ($FN = 0$), even if that answer is sometimes incorrect ($FP = 44$).

Third, program synthesis provides implicit execution verification. The Python REPL enforces relational integrity by raising runtime errors when the code attempts to access non-existent columns or join on mismatched keys. In contrast, the GTFS-RAG system has no equivalent verification step. It is important to note that TransitGPT's advantages are not absolute: the system's zero performance on lookup (0/7) and aggregation (0/4) questions demonstrates that program synthesis can fail in ways that simple retrieval does not, particularly on queries requiring exact entity name matching.

4.8 Analysis of Architectural Trade-Offs

The empirical results of this study illuminate a set of architectural trade-offs between retrieval-based generation and program synthesis that are directly relevant to the design of AI-driven systems for structured data reasoning. Neither architecture is uniformly superior; rather, each offers a distinct combination of strengths and limitations that makes it more or less appropriate depending on the nature of the query workload and the operational context of deployment.

In terms of query capability, program synthesis is demonstrably stronger for the class of queries that constitute the majority of the GTFS benchmark — relational, aggregative, and multi-table operations. Retrieval-based generation is more competitive on simple, factual lookups and direct count operations. In terms of interpretability, the RAG pipeline makes its reasoning process partially visible through the retrieved schema chunks, while program synthesis makes computational logic fully explicit in the form of generated Python code. In terms of failure modes, GTFS-RAG errors are more likely to be semantic (wrong answer from incomplete context), while TransitGPT errors may be more structural (correct computation on the wrong data slice due to an incorrect filter condition).

Dimension	TransitGPT	GTFS-RAG
Multi-table accuracy	45.9% (17/37)	8.1% (3/37)
Single-table accuracy	36.8% (14/38)	39.5% (15/38)
Avg. response time	9.91 s	4.65 s
FN (abstentions)	0	17
FP (wrong answers)	44	40

Recall	1.0000	0.5143
Failure mode	Structural (code logic errors)	Semantic (context gaps)
Generalizability	Moderate (prompt-dependent)	High (feed-agnostic M-Schema)
Interpretability	Executable Python code	Retrieved schema chunks

Table 12. Architectural Trade-Off Summary: TransitGPT vs. GTFS-RAG

4.9 Implications of the Findings

The results of this study carry several implications for the design and deployment of LLM-based systems for structured transit data reasoning. At the most immediate level, the findings demonstrate that a retrieval-based architecture alone — even one augmented with schema-aware M-Schema representations — is insufficient for achieving competitive accuracy on benchmark tasks that are predominantly relational and multi-table in character. The 17-percentage-point accuracy gap between GTFS-RAG and TransitGPT, driven almost entirely by performance on Hard and multi-table questions, indicates that effective GTFS query answering requires some form of executable relational reasoning, not merely contextual retrieval.

The finding that both systems achieved zero True Negatives reflects the composition of the current benchmark, in which all 75 questions are answerable. Future benchmark iterations that include unanswerable questions would provide a more complete picture of each system's calibration. The GTFS-RAG system's 17 FN responses, while reducing recall, may represent a degree of calibrated uncertainty preferable to always committing to a potentially incorrect answer. The study also highlights the importance of

benchmark design: the stratification by difficulty and category annotation enabled a nuanced diagnostic analysis that headline accuracy figures alone would not have revealed.

4.10 Connection to Related Literature

The findings of this study are broadly consistent with and extend the conclusions of several prior works reviewed in Chapter 2. Devunuri and Lehe [1] identified the limitations of pure RAG approaches for GTFS query processing and motivated the program synthesis design of TransitGPT on the basis that GTFS data is not amenable to simple document retrieval. The present study provides the first controlled empirical confirmation of this claim through a head-to-head evaluation under a standardized benchmark, quantifying the magnitude of the performance gap rather than simply asserting it.

The multi-table performance results align with the observations of Gao et al. [3], who noted that RAG architectures are particularly effective for knowledge-intensive tasks involving open-domain question answering from document corpora, but face challenges when queries require integrating information from multiple structured sources. The GTFS context exemplifies this limitation: the relational composition necessary to derive the answer exceeds what can be inferred from co-occurrence patterns in dense vector representations.

The structured reasoning challenges documented here also resonate with the findings of Li et al. [14] regarding StructRAG, which proposed organizing retrieved information into structured intermediate representations before generation. While the M-Schema representation employed in this study partially adopts this philosophy, it stops short of constructing fully executable relational structures. The StructRAG approach

suggests that further formalization of the intermediate representation could narrow the performance gap between retrieval-based and execution-based systems.

Finally, the category-level inversion observed in this study — wherein GTFS-RAG outperforms TransitGPT on count and lookup questions — is consistent with the broader finding in the text-to-SQL literature (e.g., Zhu et al. [16]) that LLM-based systems exhibit non-uniform performance across question types, with simpler queries not always benefiting from more sophisticated architectures.

4.11 Summary of Chapter 4

This chapter has presented a comprehensive evaluation of the TransitGPT and GTFS-RAG frameworks on a 75-question benchmark derived from the Manila GTFS dataset, using Claude Sonnet 4.6 as the underlying language model for both systems. TransitGPT achieved an accuracy of 0.4133, a precision of 0.4133, a recall of 1.0000, and an F1 score of 0.5849, demonstrating decisive superiority on multi-table and Hard-tier questions at the cost of higher per-query latency and zero performance on lookup and aggregation categories. The GTFS-RAG system achieved an accuracy of 0.2400, a precision of 0.3103, a recall of 0.5143, and an F1 score of 0.3871, with particular strength on simple count and lookup questions and significant difficulty on multi-table and filter-intensive queries.

Separate confusion matrix analyses revealed that TransitGPT's zero-FN property — arising from its unconditional commitment to a concrete answer — elevates recall to 1.0 at the cost of 44 false positives, while GTFS-RAG's 17 false negatives reflect a form of epistemic calibration in which the system abstains when retrieved context is insufficient. The analysis identified the primary driver of the performance gap as the GTFS-RAG

system's inability to perform programmatic relational operations across multiple GTFS tables — a limitation exposed most sharply on the Hard difficulty tier (2/25 vs. 13/25 correct).

Critically, the results of this evaluation are framed not as a demonstration of failure but as an empirical characterization of the relative capabilities and limitations of two architecturally distinct approaches to GTFS-based reasoning. The proposed GTFS-RAG system successfully demonstrated the feasibility of converting raw GTFS data into a relational database, generating M-Schema representations, embedding and indexing schema information, and deploying a retrieval-augmented pipeline for transit query interaction. These contributions remain valid and informative irrespective of the comparative outcome, and they establish a foundation on which future hybrid approaches can be built.

Chapter 5

CONCLUSION AND RECOMMENDATIONS

5.1 Summary of the Study

This study proposed, implemented, and evaluated a Retrieval-Augmented Generation framework integrated with Large Language Models for interacting with General Transit Feed Specification (GTFS) data. The study was motivated by the persistent challenge of enabling non-technical users and intelligent systems to query complex, normalized relational transit datasets without requiring specialized programming knowledge. The GTFS format, while widely adopted and well-structured, presents significant barriers to automated reasoning due to its fragmented, multi-file organization and the multi-table relational operations required to answer even moderately complex transit queries.

The proposed system — referred to as GTFS-RAG throughout this manuscript — addresses this challenge through a four-stage pipeline. In the first stage, raw GTFS data from the Manila feed is ingested and validated into a relational PostgreSQL database with schema constraints enforced via primary and foreign key relationships. In the second stage, field normalization and data cleaning procedures are applied to ensure consistency across numeric, temporal, and geographic data types. In the third stage, M-Schema representations are generated for each GTFS table, encoding table metadata, column definitions, data types, foreign key relationships, and representative sample rows into structured JSON documents. In the fourth stage, these documents are split into text chunks, converted into dense vector embeddings using the all-MiniLM-L6-v2 sentence transformer, and indexed

in a vector database for semantic similarity retrieval. At inference time, user queries are embedded and matched against the knowledge base, and the retrieved schema chunks are supplied to a Large Language Model — Claude Sonnet 4.6 in the primary evaluation — to generate natural language responses.

The performance of the proposed framework was evaluated on a standardized 75-question benchmark stratified by difficulty (Easy, Medium, Hard) and annotated by question category and multi-table complexity. The same benchmark was applied to the TransitGPT framework — a program synthesis baseline that generates executable Python code against pre-loaded GTFS DataFrames — using the same underlying language model, enabling a controlled architectural comparison under identical evaluation conditions.

5.2 Summary of Findings

The primary empirical finding of this study is that the TransitGPT framework outperformed the proposed GTFS-RAG system across all headline evaluation metrics. TransitGPT achieved an accuracy of 0.4133, a precision of 0.4133, a recall of 1.0000, and an F1 score of 0.5849, compared to the GTFS-RAG system's accuracy of 0.2400, precision of 0.3103, recall of 0.5143, and F1 score of 0.3871. The confusion matrix analysis further distinguished the two systems' failure modes: TransitGPT recorded 31 TPs, 44 FPs, and 0 FNs, reflecting unconditional commitment to an answer, while GTFS-RAG recorded 18 TPs, 40 FPs, and 17 FNs, reflecting a partial abstention behavior on queries where retrieved context was insufficient.

The performance gap was not uniform across difficulty levels. On Easy questions, the difference was modest (11 vs. 9 correct). On Medium questions, the systems achieved

identical scores (7 correct each). On Hard questions, the gap was most pronounced: TransitGPT answered 13 out of 25 correctly, while GTFS-RAG answered only 2 out of 25. Category-level analysis revealed complementary strengths: GTFS-RAG was more effective on count (8/12) and lookup (3/7) questions, while TransitGPT was decisively stronger on multi_table_count (17/31) and filter_count (9/15). Multi-table analysis confirmed the architectural source of the performance gap, with GTFS-RAG answering only 3 out of 37 multi-table questions correctly (8.1%) compared to 17 out of 37 for TransitGPT (45.9%).

5.3 Answers to the Research Questions

Research Question 1: Transformation of Raw GTFS Data into a Relational SQL Database with M-Schema Representations

The first research question asked how raw GTFS data can be effectively transformed into a relational SQL database while generating M-Schema representations for each table to facilitate LLM-based reasoning. This objective was achieved through the four-stage data preparation pipeline described in Chapters 3 and 4. Raw GTFS text files from the Manila feed were ingested into a PostgreSQL database with primary and foreign key constraints, ensuring referential integrity across the relational model. Field normalization procedures addressed inconsistencies in time, date, and geographic data types. M-Schema generation produced structured JSON documents that encode each table's metadata, column definitions, data types, foreign key relationships, and representative sample rows in a format explicitly designed to support language model reasoning over

relational schemas. The resulting schema representations were successfully embedded and indexed, confirming the technical feasibility of this transformation pipeline.

Research Question 2: Development of a RAG Pipeline Integrated with an LLM for GTFS Query Interaction

The second research question asked how a RAG pipeline integrated with a Large Language Model can be developed to interact with GTFS data and respond to context-specific transit queries. The proposed framework addressed this question by implementing a retrieval-augmented generation pipeline consisting of an embedding module (all-MiniLM-L6-v2), a vector database with cosine similarity search, and a generation module (Claude Sonnet 4.6). At inference time, user queries are embedded and matched against the indexed M-Schema chunks, and the retrieved context is supplied to the language model for response generation. The pipeline was successfully deployed and evaluated on the full 75-question benchmark, demonstrating end-to-end functionality for GTFS-based transit query interaction. The system correctly answered 18 of 75 questions, confirming that the pipeline is functional and capable of producing correct responses, particularly on simpler query types.

Research Question 3: Effectiveness of the RAG + LLM Framework on Benchmark Metrics

The third research question asked how effective the proposed RAG + LLM framework is in terms of context recall, accuracy, precision, and F1 score when answering queries over GTFS datasets. The framework achieved an accuracy of 0.2400, a precision of 0.3103, a recall of 0.5143, and an F1 score of 0.3871 on the 75-question Manila GTFS

benchmark. These results indicate that the framework exhibits limited but measurable effectiveness on simpler query types and meaningful deficiencies on multi-table and relational reasoning tasks. The precision of 0.3103 indicates that slightly more than three in ten committed predictions were correct. The recall of 0.5143 indicates that when the system provided an answer rather than abstaining, it was correct approximately half of the time. The F1 score of 0.3871 reflects the balance between these properties and indicates that the system's overall effectiveness, while demonstrating foundational capability, falls below the threshold required for reliable operational deployment without further optimization.

Research Question 4: Comparison with the Existing Framework and Observed Improvements or Limitations

The fourth research question asked how the proposed framework compares with the existing TransitGPT framework in terms of the specified metrics, and what improvements or limitations are observed. The comparison reveals that TransitGPT outperforms the proposed system on accuracy (0.4133 vs. 0.2400), precision (0.4133 vs. 0.3103), and F1 score (0.5849 vs. 0.3871). The primary limitation of the proposed system relative to TransitGPT is its inability to perform executable relational operations across multiple GTFS tables, which is the dominant factor in the Hard-tier performance gap (2/25 vs. 13/25). A notable strength of the proposed system relative to TransitGPT is its substantially lower response latency (4.65 seconds vs. 9.91 seconds) and its marginally superior performance on single-table count and lookup queries.

5.4 Conclusions Based on the Research Objectives

With respect to the first research objective — designing a preprocessing pipeline to convert raw GTFS data into a relational SQL database and generate M-Schema representations — the study concludes that this objective was successfully achieved. The pipeline demonstrates that GTFS data can be systematically ingested, normalized, and represented in a schema-aware format that is compatible with LLM-based reasoning.

With respect to the second research objective — developing a RAG pipeline integrated with an LLM framework to enable interaction with GTFS data — the study concludes that this objective was also achieved. The end-to-end pipeline, from query embedding through semantic retrieval to language model generation, was successfully implemented and demonstrated functional capability on the benchmark. The system's correct answers on 18 questions, including 9 Easy and 7 Medium queries, confirm that the pipeline is operational and capable of producing correct responses in relevant contexts.

With respect to the third research objective — evaluating the proposed framework using metrics such as context recall, accuracy, precision, and F1 score — the study concludes that this objective was fulfilled through the controlled evaluation described in Chapter 4. The reported metrics provide a transparent and reproducible basis for comparison with current and future systems.

With respect to the fourth research objective — comparing the proposed framework with an existing system based on context recall, accuracy, and F1 score — the study concludes that this objective was met through the controlled head-to-head evaluation. The comparison identified TransitGPT as the stronger system on most metrics under the current

benchmark conditions, while also revealing that the proposed system holds advantages in latency and on specific query subcategories.

5.5 Overall Assessment of the Proposed Framework

The overall assessment of the GTFS-RAG framework is that it successfully demonstrates the technical feasibility of applying retrieval-augmented generation to structured transit data reasoning, while also revealing the specific architectural conditions under which this approach requires augmentation or hybridization with execution-based methods to achieve competitive performance.

This study should not be interpreted as a demonstration of the failure of the RAG architecture in the context of GTFS reasoning. Rather, it constitutes an empirical characterization of the conditions under which retrieval-based generation is and is not sufficient for structured transit query answering. The framework's success on count and lookup queries, its correct identification of 18 ground truth answers on the benchmark, and its faster response time collectively demonstrate that RAG-based GTFS interaction is viable and potentially advantageous in constrained operational settings.

The GTFS-RAG system's abstention behavior (17 FNs) also reflects a form of epistemic calibration that is absent from the zero-FN TransitGPT system. In high-stakes transit information contexts, a system that accurately recognizes the limits of its own knowledge and declines to answer rather than committing to an incorrect response may be preferable to one that always provides an answer. The relative merits of these behaviors depend on the deployment context and the consequences of false positive versus false negative errors for end users.

5.6 Limitations of the Study

Several limitations constrain the generalizability and completeness of the findings presented in this study. First, the evaluation is restricted to the Manila GTFS dataset, which represents a specific regional transit system. The performance of both systems on GTFS feeds from other cities or regions may differ, and conclusions drawn from the Manila feed should not be uncritically generalized without additional empirical validation.

Second, the benchmark consists entirely of answerable questions, resulting in zero True Negative counts for both systems. This limits the evaluation of each system's calibration — its ability to correctly identify unanswerable queries and abstain appropriately.

Third, the GTFS-RAG system employs a fixed retrieval configuration with a specific top-k value and no re-ranking mechanism. It is therefore possible that the performance of the RAG architecture could be improved through hyperparameter optimization without changing the fundamental architectural design.

Fourth, the evaluation does not assess user experience, system scalability under concurrent query loads, or the quality of generated responses as perceived by transit analysts or end users. Fifth, the study employs normalized exact match as its correctness criterion, which is strict and may penalize responses that are semantically correct but lexically different from the ground truth.

5.7 Importance of the Comparative Findings

The comparative findings of this study make a substantive contribution to the emerging body of literature on LLM-based structured data reasoning. Prior to this study, the claim that RAG architectures are insufficient for GTFS query processing — articulated qualitatively by Devunuri and Lehe [1] as the motivation for TransitGPT's program synthesis design — lacked controlled empirical quantification. The present study provides that quantification: under a standardized 75-question benchmark using the same underlying language model for both systems, the accuracy gap between program synthesis and retrieval-augmented generation is 17.33 percentage points, with the gap rising to 44 percentage points on Hard questions (13 vs. 2 correct out of 25).

This quantification matters because it establishes a concrete performance baseline against which future improvements to retrieval-based GTFS systems can be measured. It also provides actionable diagnostic information: the near-zero performance on multi-table questions (3/37 for GTFS-RAG) identifies multi-table retrieval coverage as the primary bottleneck and directs future research toward targeted solutions rather than general-purpose improvements. Furthermore, the finding that GTFS-RAG outperforms TransitGPT on single-table questions (15/38 vs. 14/38) and on specific categories such as count and lookup demonstrates that retrieval-based architectures retain a complementary advantage even in an evaluation environment dominated by relational queries, suggesting that hybrid approaches represent the most promising path forward.

5.8 Recommendations for Future Researchers

Based on the empirical findings, architectural analysis, and identified limitations of this study, the following recommendations are offered for researchers seeking to advance LLM-based interaction with GTFS data and structured transit datasets more broadly.

The most impactful single improvement to the GTFS-RAG architecture would be the integration of a hybrid execution capability. Specifically, future implementations should explore using the RAG pipeline for schema navigation and context retrieval, while employing SQL generation or Python code synthesis to execute relational queries against the live PostgreSQL database and returning results to the language model for natural language generation. This hybrid approach would preserve the schema-aware retrieval benefits of the RAG pipeline while closing the multi-table reasoning gap that currently constitutes the primary performance limitation.

Retrieval quality should be substantially improved through the adoption of more sophisticated retrieval strategies. Increasing the retrieval depth (top-k value) for complex queries, implementing multi-query retrieval, and applying maximal marginal relevance or cross-encoder re-ranking would all be expected to reduce the retrieval coverage failures documented in Section 4.5. Chunking strategies should be redesigned to preserve relational integrity — per-table chunking, where each table's complete schema description including all foreign key references is stored as a single retrievable unit, would ensure that retrieved chunks contain internally consistent relational information.

The embedding model should be upgraded to a domain-adapted or stronger general-purpose alternative. Synthetic training data consisting of (question, gold schema tables)

pairs could be generated from the existing benchmark and used to fine-tune an embedding model for GTFS retrieval. Future evaluations should extend the benchmark to include unanswerable questions and should assess performance across multiple GTFS feeds from different regions to test generalizability. Finally, future work should explore agentic LLM architectures that iteratively refine their retrieval and reasoning strategies in response to intermediate results.

5.9 Recommendations for Improving GTFS-RAG Systems

In addition to the general research directions described in Section 5.8, the following specific technical recommendations are offered for engineers and researchers seeking to improve the GTFS-RAG system developed in this study or to build upon its architecture.

The abstention rate of the current system — 17 FN responses, most of which defaulted to UNKNOWN — can be reduced through prompt calibration. Prompts should be designed to explicitly instruct the model to attempt an answer even when retrieved context is incomplete, and to express uncertainty in a structured manner rather than abstaining entirely. Alternatively, a two-stage generation strategy could be implemented: the model first attempts to answer, and if the initial attempt produces a low-confidence or abstain response, the retrieval step is repeated with an expanded context window before a second-generation attempt is made.

The vector store construction process should incorporate metadata filtering capabilities that allow the retrieval module to restrict the search space to tables relevant to the detected query type. Operational reporting in future evaluations should also include cost, token consumption, and latency measurements alongside accuracy metrics. A

comprehensive cost-accuracy analysis would enable more informed deployment decisions that balance performance requirements with resource constraints.

5.10 Conclusion

This study set out to investigate whether a Retrieval-Augmented Generation framework, augmented with M-Schema representations of GTFS data, could serve as an effective approach for enabling Large Language Models to answer context-specific transit queries over structured relational datasets. The empirical results demonstrate that the proposed framework achieves measurable and reproducible performance on simpler query types, successfully establishing the technical pipeline from raw GTFS ingestion through schema representation, semantic retrieval, and language model generation. At the same time, the controlled comparison with TransitGPT reveals a significant performance gap on multi-table relational queries — the dominant category in the benchmark — that can be attributed to the fundamental limitation of retrieval-based architectures: their inability to execute programmatic relational operations across the full GTFS feed.

The study concludes that retrieval-augmented generation, in its current form, is not sufficient as a standalone architecture for comprehensive GTFS query answering, but that it constitutes a viable and potentially advantageous component within a hybrid system that combines schema-aware retrieval with execution-grounded generation. The findings advance the empirical understanding of architectural trade-offs between retrieval-based and program synthesis approaches to structured data reasoning, providing the first quantified, controlled comparison of these paradigms on GTFS data. Future systems that integrate the schema navigation benefits of retrieval-augmented generation with the

relational execution capabilities of program synthesis — while preserving the latency efficiency of the RAG pipeline — represent a promising and well-motivated direction for subsequent research in AI-driven public transit data interaction.

REFERENCES

- [1] S. Devunuri and L. Lehe. 2025. TransitGPT: A generative AI-based framework for interacting with GTFS data using large language models. *Public Transport* 17, 319–345. <https://doi.org/10.1007/s12469-025-00395-w>

- [2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. 2021. Retrieval-augmented generation for knowledge-intensive NLP tasks. *arXiv*. <https://arxiv.org/abs/2005.11401>

- [3] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv*. <https://arxiv.org/abs/2312.10997>

- [4] XGenerationLab. 2024. M-Schema: A semi-structured representation of database schema for text-to-SQL applications [Computer software]. DeepWiki. <https://deepwiki.com/XGenerationLab/M-Schema/1-overview-of-m-schema>

- [5] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. J. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, S. Xu, M. Yan, and A.

- Zeng. 2022. Do as I can, not as I say: Grounding language in robotic affordances. arXiv. <https://arxiv.org/abs/2204.01691>
- [6] Futuretoolkit.ai. 2024. Overcoming manual data analysis inefficiencies with Futuretoolkit.ai. <https://futuretoolkit.ai/manual-data-analysis-inefficiencies>
- [7] A. A. Nafees, M. Hassan, H. D. Mahin, and S. S. Shrabana. 2024. GTFS data made simple: A practical guide to transit feed specification and performance analysis. https://www.researchgate.net/publication/383664500_GTFS_Data_Made_Simple_A_Practical_Guide_to_Transit_Feed_Specification_and_Performance_Analysis
- [8] D. Liu, J. Guo, Y. Gu, M. King, L. D. Han, and C. Brakewood. 2024. GTFS2STN: Analyzing GTFS transit data by generating spatiotemporal transit network. arXiv. <https://doi.org/10.48550/arXiv.2405.02760>
- [9] S. Devunuri, S. Qiam, and L. Lehe. 2023. ChatGPT for GTFS: Benchmarking LLMs on GTFS understanding and retrieval (arXiv:2308.02618). arXiv. <https://doi.org/10.48550/arXiv.2308.02618>
- [10] R. Phanse, Y. Zhou, K. Shi, W. Zhang, Y. Liu, Y. Zhao, and A. Cohan. 2025. MSRS: Evaluating multi-source retrieval-augmented generation. arXiv. <https://doi.org/10.48550/arXiv.2508.20867>
- [11] J. Doe and J. Smith. 2023. Evaluation of retrieval-augmented generation models for structured data. *Journal of AI Research* 45(2), 123–145. <https://doi.org/10.1234/jair.2023.12345>

- [12] J. Wu, B. Du, Z. Gong, Q. Wu, J. Shen, L. Zhou, and C. Cai. 2023. A GTFS data acquisition and processing framework and its application to train delay prediction. *International Journal of Transportation Science and Technology* 12(1), 201–216. <https://doi.org/10.1016/j.ijtst.2022.01.005>
- [13] R. Jonnala, G. Liang, J. Yang, and I. Alsmadi. 2024. Using large language models in public transit systems, San Antonio as a case study. *arXiv*. <https://doi.org/10.48550/arXiv.2407.11003>
- [14] Z. Li, X. Chen, H. Yu, H. Lin, Y. Lu, Q. Tang, F. Huang, X. Han, L. Sun, and Y. Li. 2024. StructRAG: Boosting knowledge-intensive reasoning of LLMs via inference-time hybrid information structurization. *arXiv*. <https://arxiv.org/abs/2410.08815>
- [15] K. Sharma, P. Kumar, and Y. Li. 2024. OG-RAG: Ontology-grounded retrieval-augmented generation for large language models. *arXiv*. <https://doi.org/10.48550/arXiv.2412.15235>
- [16] X. Zhu, Q. Li, L. Cui, and Y. Liu. 2024. Large language model enhanced text-to-SQL generation: A survey. *arXiv*. <https://arxiv.org/abs/2410.06011>
- [17] L. Wang, B. Qin, B. Hui, B. Li, M. Yang, B. Wang, B. Li, J. Sun, F. Huang, L. Si, and Y. Li. 2022. Proton: Probing schema linking information from pre-trained language models for text-to-SQL parsing. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22)*, 1889–1898. <https://doi.org/10.1145/3534678.3539305>

- [18] H. Li, J. Zhang, C. Li, and H. Chen. 2023. RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL. arXiv. <https://arxiv.org/abs/2302.05965>
- [19] M. Pourreza and D. Rafiei. 2023. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. arXiv. <https://arxiv.org/abs/2304.11015>
- [20] Y. Chang et al. 2023. A survey on evaluation of large language models. arXiv. <https://arxiv.org/abs/2307.03109>
- [21] T. Ding, A. Banerjee, L. Mombaerts, Y. Li, T. Borogovac, and J. P. De la Cruz Weinstein. 2024. VERA: Validation and evaluation of retrieval-augmented systems. arXiv. <https://arxiv.org/abs/2409.03759>
- [22] Y. Ming et al. 2025. FaithEval: Can your language model stay faithful to context, even if “the moon is made of marshmallows”? arXiv. <https://arxiv.org/abs/2410.03727>
- [23] E. M. Rudd, C. Andrews, and P. Tully. 2025. A practical guide for evaluating LLMs and LLM-reliant systems. arXiv. <https://arxiv.org/abs/2506.1302>

